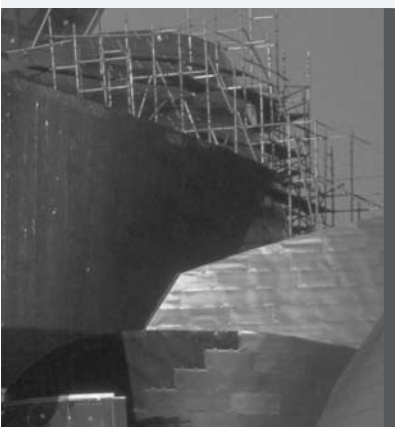




PARTE

1

Introducción a la ingeniería de software

La meta de esta parte del libro es ofrecer una introducción general a la ingeniería de software. Se incluyen conceptos importantes como procesos de software y métodos ágiles; además, se describen actividades esenciales del desarrollo de software, desde la especificación inicial del software hasta la evolución del sistema. Los capítulos de esta parte se diseñaron para apoyar un curso de un semestre en ingeniería de software.

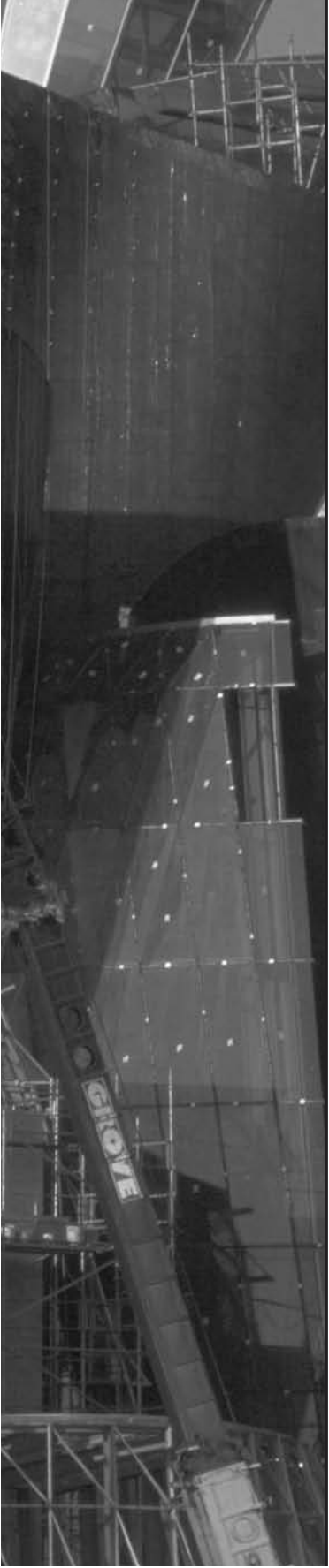
El capítulo 1 introduce al lector de manera general en la ingeniería de software profesional y define algunos conceptos al respecto. También se escribe un breve análisis de los conflictos éticos en la ingeniería de software. Es importante que los ingenieros de software consideren las numerosas implicaciones de su trabajo. Este capítulo además presenta tres estudios de caso que se usan en el libro, a saber: un sistema para administrar registros de pacientes que se someten a tratamiento por problemas de salud mental, un sistema de control para una bomba de insulina portátil y un sistema meteorológico a campo abierto.

Los capítulos 2 y 3 tratan los procesos de ingeniería de software y el desarrollo ágil. En el capítulo 2 se presentan modelos de proceso de software genérico de uso común, como el waterfall (cascada), y se estudian las actividades básicas que son parte de dichos procesos. El capítulo 3 complementa esto con un estudio de los métodos de desarrollo ágil para ingeniería de software. Se usa sobre todo la programación extrema como

ejemplo de un método ágil, aunque también en esta sección se introduce brevemente Scrum.

Los capítulos restantes son amplias descripciones de las actividades del proceso de software que se introducirán en el capítulo 2. En el capítulo 4 se trata un tema importante de la ingeniería de requerimientos, donde se definen las necesidades de lo que debe hacer un sistema. El capítulo 5 muestra el modelado de sistemas usando el UML, donde se enfoca el uso de los diagramas de caso, diagramas de clase, diagramas de secuencia y diagramas de estado para modelar un sistema de software. El capítulo 6 introduce al diseño arquitectónico y estudia la importancia de la arquitectura y el uso de patrones arquitectónicos en el diseño de software.

El capítulo 7 trata sobre el diseño orientado a objetos y el uso de patrones de diseño. Aquí también se observan importantes problemas de implementación: reutilización, manejo de configuración y el desarrollo de host-target (anfitrión destino); también se estudia el desarrollo de código abierto. El capítulo 8 se enfoca en las pruebas del software, desde la prueba de unidad durante el desarrollo del sistema, hasta la prueba de la puesta en venta del software. También se analiza el uso del desarrollo impulsado por prueba, un enfoque pionero en los métodos ágiles, pero con gran aplicabilidad. Finalmente, el capítulo 9 brinda un panorama de los temas sobre la evolución del software. Se describen los procesos evolutivos, el mantenimiento del software y la gestión de sistemas legados.



1

Introducción

Objetivos

Los objetivos de este capítulo consisten en introducir al lector a la ingeniería de software y ofrecer un marco conceptual para entender el resto del libro. Al estudiar este capítulo:

- conocerá qué es la ingeniería de software y por qué es importante;
- comprenderá que el desarrollo de diferentes tipos de sistemas de software puede requerir distintas técnicas de ingeniería de software;
- entenderá algunos conflictos éticos y profesionales que son importantes para los ingenieros de software;
- conocerá tres sistemas de diferentes tipos, que se usarán como ejemplos a lo largo del libro.

Contenido

- 1.1** Desarrollo de software profesional
- 1.2** Ética en la ingeniería de software
- 1.3** Estudios de caso

Es imposible operar el mundo moderno sin software. Las infraestructuras nacionales y los servicios públicos se controlan mediante sistemas basados en computadoras, y la mayoría de los productos eléctricos incluyen una computadora y un software de control. La fabricación y la distribución industrial están completamente computarizadas, como el sistema financiero. El entretenimiento, incluida la industria musical, los juegos por computadora, el cine y la televisión, usan software de manera intensiva. Por lo tanto, la ingeniería de software es esencial para el funcionamiento de las sociedades, tanto a nivel nacional como internacional.

Los sistemas de software son abstractos e intangibles. No están restringidos por las propiedades de los materiales, regidos por leyes físicas ni por procesos de fabricación. Esto simplifica la ingeniería de software, pues no existen límites naturales a su potencial. Sin embargo, debido a la falta de restricciones físicas, los sistemas de software pueden volverse rápidamente muy complejos, difíciles de entender y costosos de cambiar.

Hay muchos tipos diferentes de sistemas de software, desde los simples sistemas embebidos, hasta los complejos sistemas de información mundial. No tiene sentido buscar notaciones, métodos o técnicas universales para la ingeniería de software, ya que diferentes tipos de software requieren distintos enfoques. Desarrollar un sistema organizacional de información es completamente diferente de un controlador para un instrumento científico. Ninguno de estos sistemas tiene mucho en común con un juego por computadora de gráficos intensivos. Aunque todas estas aplicaciones necesitan ingeniería de software, no todas requieren las mismas técnicas de ingeniería de software.

Aún existen muchos reportes tanto de proyectos de software que salen mal como de “fallas de software”. Por ello, a la ingeniería de software se le considera inadecuada para el desarrollo del software moderno. Sin embargo, desde la perspectiva del autor, muchas de las llamadas fallas del software son consecuencia de dos factores:

1. *Demandas crecientes* Conforme las nuevas técnicas de ingeniería de software ayudan a construir sistemas más grandes y complejos, las demandas cambian. Los sistemas tienen que construirse y distribuirse más rápidamente; se requieren sistemas más grandes e incluso más complejos; los sistemas deben tener nuevas capacidades que anteriormente se consideraban imposibles. Los métodos existentes de ingeniería de software no pueden enfrentar la situación, y tienen que desarrollarse nuevas técnicas de ingeniería de software para satisfacer nuevas demandas.
2. *Expectativas bajas* Es relativamente sencillo escribir programas de cómputo sin usar métodos y técnicas de ingeniería de software. Muchas compañías se deslizan hacia la ingeniería de software conforme evolucionan sus productos y servicios. No usan métodos de ingeniería de software en su trabajo diario. Por lo tanto, su software con frecuencia es más costoso y menos confiable de lo que debiera. Es necesaria una mejor educación y capacitación en ingeniería de software para solucionar este problema.

Los ingenieros de software pueden estar orgullosos de sus logros. Desde luego, todavía se presentan problemas al desarrollar software complejo, pero, sin ingeniería de software, no se habría explorado el espacio ni se tendría Internet o las telecomunicaciones modernas. Todas las formas de viaje serían más peligrosas y caras. La ingeniería de software ha contribuido en gran medida, y sus aportaciones en el siglo XXI serán aún mayores.



Historia de la ingeniería de software

El concepto “ingeniería de software” se propuso originalmente en 1968, en una conferencia realizada para discutir lo que entonces se llamaba la “crisis del software” (Naur y Randell, 1969). Se volvió claro que los enfoques individuales al desarrollo de programas no escalaban hacia los grandes y complejos sistemas de software. Éstos no eran confiables, costaban más de lo esperado y se distribuían con demora.

A lo largo de las décadas de 1970 y 1980 se desarrolló una variedad de nuevas técnicas y métodos de ingeniería de software, tales como la programación estructurada, el encubrimiento de información y el desarrollo orientado a objetos. Se perfeccionaron herramientas y notaciones estándar y ahora se usan de manera extensa.

<http://www.SoftwareEngineering-9.com/Web/History/>

1.1 Desarrollo de software profesional

Muchos individuos escriben programas. En las empresas los empleados hacen programas de hoja de cálculo para simplificar su trabajo; científicos e ingenieros elaboran programas para procesar sus datos experimentales, y los aficionados crean programas para su propio interés y satisfacción. Sin embargo, la gran mayoría del desarrollo de software es una actividad profesional, donde el software se realiza para propósitos de negocios específicos, para su inclusión en otros dispositivos o como productos de software, por ejemplo, sistemas de información, sistemas de CAD, etcétera. El software profesional, destinado a usarse por alguien más aparte de su desarrollador, se lleva a cabo en general por equipos, en vez de individualmente. Se mantiene y cambia a lo largo de su vida.

La ingeniería de software busca apoyar el desarrollo de software profesional, en lugar de la programación individual. Incluye técnicas que apoyan la especificación, el diseño y la evolución del programa, ninguno de los cuales son normalmente relevantes para el desarrollo de software personal. Con el objetivo de ayudarlo a obtener una amplia visión de lo que trata la ingeniería de software, en la figura 1.1 se resumen algunas preguntas planteadas con frecuencia.

Muchos suponen que el software es tan sólo otra palabra para los programas de cómputo. No obstante, cuando se habla de ingeniería de software, esto no sólo se refiere a los programas en sí, sino también a toda la documentación asociada y los datos de configuración requeridos para hacer que estos programas operen de manera correcta. Un sistema de software desarrollado profesionalmente es usualmente más que un solo programa. El sistema por lo regular consta de un número de programas separados y archivos de configuración que se usan para instalar dichos programas. Puede incluir documentación del sistema, que describe la estructura del sistema; documentación del usuario, que explica cómo usar el sistema, y los sitios web para que los usuarios descarguen información reciente del producto.

Ésta es una de las principales diferencias entre el desarrollo de software profesional y el de aficionado. Si usted diseña un programa personal, nadie más lo usará ni tendrá que preocuparse por elaborar guías del programa, documentar el diseño del programa, etcétera. Por el contrario, si crea software que otros usarán y otros ingenieros cambiarán, entonces, en general debe ofrecer información adicional, así como el código del programa.

Pregunta	Respuesta
¿Qué es software?	Programas de cómputo y documentación asociada. Los productos de software se desarrollan para un cliente en particular o para un mercado en general.
¿Cuáles son los atributos del buen software?	El buen software debe entregar al usuario la funcionalidad y el desempeño requeridos, y debe ser sustentable, confiable y utilizable.
¿Qué es ingeniería de software?	La ingeniería de software es una disciplina de la ingeniería que se interesa por todos los aspectos de la producción de software.
¿Cuáles son las actividades fundamentales de la ingeniería de software?	Especificación, desarrollo, validación y evolución del software.
¿Cuál es la diferencia entre ingeniería de software y ciencias de la computación?	Las ciencias de la computación se enfocan en teoría y fundamentos; mientras la ingeniería de software se enfoca en el sentido práctico del desarrollo y en la distribución de software.
¿Cuál es la diferencia entre ingeniería de software e ingeniería de sistemas?	La ingeniería de sistemas se interesa por todos los aspectos del desarrollo de sistemas basados en computadoras, incluidos hardware, software e ingeniería de procesos. La ingeniería de software es parte de este proceso más general.
¿Cuáles son los principales retos que enfrenta la ingeniería de software?	Se enfrentan con una diversidad creciente, demandas por tiempos de distribución limitados y desarrollo de software confiable.
¿Cuáles son los costos de la ingeniería de software?	Aproximadamente 60% de los costos del software son de desarrollo, y 40% de prueba. Para el software elaborado específicamente, los costos de evolución superan con frecuencia los costos de desarrollo.
¿Cuáles son los mejores métodos y técnicas de la ingeniería de software?	Aun cuando todos los proyectos de software deben gestionarse y desarrollarse de manera profesional, existen diferentes técnicas que son adecuadas para distintos tipos de sistema. Por ejemplo, los juegos siempre deben diseñarse usando una serie de prototipos, mientras que los sistemas críticos de control de seguridad requieren de una especificación completa y analizable para su desarrollo. Por lo tanto, no puede decirse que un método sea mejor que otro.
¿Qué diferencias ha marcado la Web a la ingeniería de software?	La Web ha llevado a la disponibilidad de servicios de software y a la posibilidad de desarrollar sistemas basados en servicios distribuidos ampliamente. El desarrollo de sistemas basados en Web ha conducido a importantes avances en lenguajes de programación y reutilización de software.

Figura 1.1 Preguntas planteadas con frecuencia sobre el software

Los ingenieros de software están interesados por el desarrollo de productos de software (es decir, software que puede venderse a un cliente). Existen dos tipos de productos de software:

1. *Productos genéricos* Consisten en sistemas independientes que se producen por una organización de desarrollo y se venden en el mercado abierto a cualquier cliente

que desee comprarlos. Ejemplos de este tipo de productos incluyen software para PC, como bases de datos, procesadores de texto, paquetes de dibujo y herramientas de administración de proyectos. También abarcan las llamadas aplicaciones verticales diseñadas para cierto propósito específico, tales como sistemas de información de librería, sistemas de contabilidad o sistemas para mantener registros dentales.

2. *Productos personalizados (o a la medida)* Son sistemas que están destinados para un cliente en particular. Un contratista de software desarrolla el programa especialmente para dicho cliente. Ejemplos de este tipo de software incluyen los sistemas de control para dispositivos electrónicos, sistemas escritos para apoyar cierto proceso empresarial y los sistemas de control de tráfico aéreo.

Una diferencia importante entre estos tipos de software es que, en productos genéricos, la organización que desarrolla el software controla la especificación del mismo. Para los productos personalizados, la organización que compra el software generalmente desarrolla y controla la especificación, por lo que los desarrolladores de software deben trabajar siguiendo dicha especificación.

Sin embargo, la distinción entre estos tipos de producto de sistemas se vuelve cada vez más difusa. Ahora, cada vez más sistemas se construyen con un producto genérico como base, que luego se adapta para ajustarse a los requerimientos de un cliente. Los sistemas Enterprise Resource Planning (ERP, planeación de recursos empresariales), como el sistema SAP, son los mejores ejemplos de este enfoque. Aquí, un sistema grande y complejo se adapta a una compañía al incorporar la información acerca de las reglas y los procesos empresariales, los reportes requeridos, etcétera.

Cuando se habla de la calidad del software profesional, se debe considerar que el software lo usan y cambian personas, además de sus desarrolladores. En consecuencia, la calidad no tiene que ver sólo con lo que hace el software. En cambio, debe incluir el comportamiento del software mientras se ejecuta, y la estructura y organización de los programas del sistema y la documentación asociada. Esto se refleja en los llamados calidad o atributos no funcionales del software. Ejemplos de dichos atributos son el tiempo de respuesta del software ante la duda de un usuario y la comprensibilidad del código del programa.

El conjunto específico de atributos que se espera de un sistema de software depende evidentemente de su aplicación. Así, un sistema bancario debe ser seguro, un juego interactivo debe tener capacidad de respuesta, un sistema de conmutación telefónica debe ser confiable, etcétera. Esto puede generalizarse en el conjunto de atributos que se muestra en la figura 1.2, los cuales consideran las características esenciales de un sistema de software profesional.

1.1.1 Ingeniería de software

La ingeniería de software es una disciplina de ingeniería que se interesa por todos los aspectos de la producción de software, desde las primeras etapas de la especificación del sistema hasta el mantenimiento del sistema después de que se pone en operación. En esta definición se presentan dos frases clave:

1. *Disciplina de ingeniería* Los ingenieros hacen que las cosas funcionen. Aplican teorías, métodos y herramientas donde es adecuado. Sin embargo, los usan de manera

Características del producto	Descripción
Mantenimiento	El software debe escribirse de tal forma que pueda evolucionar para satisfacer las necesidades cambiantes de los clientes. Éste es un atributo crítico porque el cambio del software es un requerimiento inevitable de un entorno empresarial variable.
Confiabilidad y seguridad	La confiabilidad del software incluye un rango de características que abarcan fiabilidad, seguridad y protección. El software confiable no tiene que causar daño físico ni económico, en caso de falla del sistema. Los usuarios malintencionados no deben tener posibilidad de acceder al sistema o dañarlo.
Eficiencia	El software no tiene que desperdiciar los recursos del sistema, como la memoria y los ciclos del procesador. Por lo tanto, la eficiencia incluye capacidad de respuesta, tiempo de procesamiento, utilización de memoria, etcétera.
Aceptabilidad	El software debe ser aceptable al tipo de usuarios para quienes se diseña. Esto significa que necesita ser comprensible, utilizable y compatible con otros sistemas que ellos usan.

Figura 1.2 Atributos esenciales del buen software

selectiva y siempre tratan de encontrar soluciones a problemas, incluso cuando no hay teorías ni métodos aplicables. Los ingenieros también reconocen que deben trabajar ante restricciones organizacionales y financieras, de modo que buscan soluciones dentro de tales limitaciones.

2. *Todos los aspectos de la producción del software* La ingeniería de software no sólo se interesa por los procesos técnicos del desarrollo de software, sino también incluye actividades como la administración del proyecto de software y el desarrollo de herramientas, así como métodos y teorías para apoyar la producción de software.

La ingeniería busca obtener resultados de la calidad requerida dentro de la fecha y del presupuesto. A menudo esto requiere contraer compromisos: los ingenieros no deben ser perfeccionistas. Sin embargo, las personas que diseñan programas para sí mismas podrían pasar tanto tiempo como deseen en el desarrollo del programa.

En general, los ingenieros de software adoptan en su trabajo un enfoque sistemático y organizado, pues usualmente ésta es la forma más efectiva de producir software de alta calidad. No obstante, la ingeniería busca seleccionar el método más adecuado para un conjunto de circunstancias y, de esta manera, un acercamiento al desarrollo más creativo y menos formal sería efectivo en ciertas situaciones. El desarrollo menos formal es particularmente adecuado para la creación de sistemas basados en la Web, que requieren una mezcla de habilidades de software y diseño gráfico.

La ingeniería de software es importante por dos razones:

1. Cada vez con mayor frecuencia, los individuos y la sociedad se apoyan en los avanzados sistemas de software. Por ende, se requiere producir económica y rápidamente sistemas confiables.

2. A menudo resulta más barato a largo plazo usar métodos y técnicas de ingeniería de software para los sistemas de software, que sólo diseñar los programas como si fuera un proyecto de programación personal. Para muchos tipos de sistemas, la mayoría de los costos consisten en cambiar el software después de ponerlo en operación.

El enfoque sistemático que se usa en la ingeniería de software se conoce en ocasiones como proceso de software. Un proceso de software es una secuencia de actividades que conducen a la elaboración de un producto de software. Existen cuatro actividades fundamentales que son comunes a todos los procesos de software, y éstas son:

1. Especificación del software, donde clientes e ingenieros definen el software que se producirá y las restricciones en su operación.
2. Desarrollo del software, donde se diseña y programa el software.
3. Validación del software, donde se verifica el software para asegurar que sea lo que el cliente requiere.
4. Evolución del software, donde se modifica el software para reflejar los requerimientos cambiantes del cliente y del mercado.

Diferentes tipos de sistemas necesitan distintos procesos de desarrollo. Por ejemplo, el software en tiempo real en una aeronave debe especificarse por completo antes de comenzar el desarrollo. En los sistemas de comercio electrónico, la especificación y el programa por lo general se desarrollan en conjunto. En consecuencia, tales actividades genéricas pueden organizarse en diferentes formas y describirse en distintos niveles de detalle, dependiendo del tipo de software que se vaya a desarrollar. En el capítulo 2 se describen con más puntualidad los procesos de software.

La ingeniería de software se relaciona con las ciencias de la computación y la ingeniería de sistemas:

1. Las ciencias de la computación se interesan por las teorías y los métodos que subyacen en las computadoras y los sistemas de software, en tanto que la ingeniería de software se preocupa por los asuntos prácticos de la producción del software. Ciertamente, el conocimiento de ciencias de la computación es esencial para los ingenieros de software, del mismo modo que cierto conocimiento de física lo es para los ingenieros electricistas. Sin embargo, con frecuencia la teoría de las ciencias de la computación es más aplicable a programas relativamente pequeños. Las teorías de las ciencias de la computación no siempre pueden aplicarse a grandes problemas complejos que requieren una solución de software.
2. La ingeniería de sistemas se interesa por todos los aspectos del desarrollo y la evolución de complejos sistemas, donde el software tiene un papel principal. Por lo tanto, la ingeniería de sistemas se preocupa por el desarrollo de hardware, el diseño de políticas y procesos, la implementación del sistema, así como por la ingeniería de software. Los ingenieros de sistemas intervienen en la especificación del sistema, definiendo su arquitectura global y, luego, integrando las diferentes partes para crear el sistema terminado. Están menos preocupados por la ingeniería de los componentes del sistema (hardware, software, etcétera).

Como se expone en la siguiente sección, hay muchos tipos diferentes de software. No existe un método o una técnica universales en la ingeniería de software que sea aplicable para todos éstos. No obstante, tres problemas generales afectan a muy diversos tipos de software:

1. *Heterogeneidad* Cada vez con mayor frecuencia se requieren sistemas que operen como distribuidos a través de redes que incluyan diferentes tipos de computadoras y dispositivos móviles. Es posible que el software se ejecute tanto en computadoras de propósito general como en teléfonos móviles. Se tendrá que integrar con frecuencia el nuevo software con sistemas legados más viejos, escritos en diferentes lenguajes de programación. El reto aquí es desarrollar técnicas para construir software confiable que sea suficientemente flexible para enfrentar esa heterogeneidad.
2. *Cambio empresarial y social* Los negocios y la sociedad cambian de manera increíblemente rápida, conforme se desarrollan las economías emergentes y nuevas tecnologías están a la disposición. Ambos necesitan tener la posibilidad de cambiar su software existente y desarrollar rápidamente uno nuevo. Muchas técnicas tradicionales de la ingeniería de software consumen tiempo, y generalmente la entrega de los nuevos sistemas tarda más de lo planeado. Requieren evolucionar de modo que se reduzca el tiempo necesario para que el software dé valor a sus clientes.
3. *Seguridad y confianza* Dado que el software está vinculado con todos los aspectos de la vida, es esencial confiar en dicho software. Esto es especialmente cierto para los sistemas de software remoto a los que se accede a través de una página Web o una interfaz de servicio Web. Es necesario asegurarse de que usuarios malintencionados no puedan atacar el software y que se conserve la seguridad de la información.

Desde luego, éstos no son problemas independientes. Por ejemplo, quizá sea necesario realizar cambios rápidos a un sistema legado con la finalidad de dotarlo con una interfaz de servicio Web. Para enfrentar dichos retos se necesitarán nuevas herramientas y técnicas, así como formas innovadoras de combinar y usar los métodos existentes de ingeniería de software.

1.1.2 Diversidad de la ingeniería de software

La ingeniería de software es un enfoque sistemático para la producción de software que toma en cuenta los temas prácticos de costo, fecha y confiabilidad, así como las necesidades de clientes y fabricantes de software. Como este enfoque sistemático realmente implementado varía de manera drástica dependiendo de la organización que desarrolla el software, el tipo de software y los individuos que intervienen en el proceso de desarrollo, no existen métodos y técnicas universales de ingeniería de software que sean adecuados para todos los sistemas y las compañías. Más bien, durante los últimos 50 años evolucionó un conjunto de métodos y herramientas de ingeniería de software.

Quizás el factor más significativo en la determinación de qué métodos y técnicas de la ingeniería de software son más importantes, es el tipo de aplicación que está siendo desarrollada. Existen muchos diferentes tipos de aplicación, incluidos los siguientes:

1. *Aplicaciones independientes* Se trata de sistemas de aplicación que corren en una computadora local, como una PC, e incluyen toda la funcionalidad necesaria

y no requieren conectarse a una red. Ejemplos de tales aplicaciones son las de oficina en una PC, programas CAD, software de manipulación de fotografías, etcétera.

2. *Aplicaciones interactivas basadas en transacción* Consisten en aplicaciones que se ejecutan en una computadora remota y a las que los usuarios acceden desde sus propias PC o terminales. Evidentemente, en ellas se incluyen aplicaciones Web como las de comercio electrónico, donde es posible interactuar con un sistema remoto para comprar bienes y servicios. Esta clase de aplicación también incluye sistemas empresariales, donde una organización brinda acceso a sus sistemas a través de un navegador Web o un programa de cliente de propósito específico y servicios basados en nube, como correo electrónico y compartición de fotografías. Las aplicaciones interactivas incorporan con frecuencia un gran almacén de datos al que se accede y actualiza en cada transacción.
3. *Sistemas de control embebido* Se trata de sistemas de control de software que regulan y gestionan dispositivos de hardware. Numéricamente, quizás existen más sistemas embebidos que cualquier otro tipo de sistema. Algunos ejemplos de sistemas embebidos incluyen el software en un teléfono móvil (celular), el software que controla los frenos antibloqueo de un automóvil y el software en un horno de microondas para controlar el proceso de cocinado.
4. *Sistemas de procesamiento en lotes* Son sistemas empresariales que se diseñan para procesar datos en grandes lotes (batch). Procesan gran cantidad de entradas individuales para crear salidas correspondientes. Los ejemplos de sistemas batch incluyen sistemas de facturación periódica, como los sistemas de facturación telefónica y los sistemas de pago de salario.
5. *Sistemas de entretenimiento* Son sistemas para uso sobre todo personal, que tienen la intención de entretener al usuario. La mayoría de estos sistemas son juegos de uno u otro tipo. La calidad de interacción ofrecida al usuario es la característica más importante de los sistemas de entretenimiento.
6. *Sistemas para modelado y simulación* Éstos son sistemas que desarrollan científicos e ingenieros para modelar procesos o situaciones físicas, que incluyen muchos objetos separados interactuantes. Dichos sistemas a menudo son computacionalmente intensivos y para su ejecución requieren sistemas paralelos de alto desempeño.
7. *Sistemas de adquisición de datos* Son sistemas que desde su entorno recopilan datos usando un conjunto de sensores, y envían dichos datos para su procesamiento a otros sistemas. El software tiene que interactuar con los sensores y se instala regularmente en un ambiente hostil, como en el interior de un motor o en una ubicación remota.
8. *Sistemas de sistemas* Son sistemas compuestos de un cierto número de sistemas de software. Algunos de ellos son producto del software genérico, como un programa de hoja de cálculo. Otros sistemas en el ensamble pueden estar especialmente escritos para ese entorno.

Desde luego, los límites entre estos tipos de sistemas son difusos. Si se desarrolla un juego para un teléfono móvil (celular), se debe tomar en cuenta las mismas restricciones (energía, interacción de hardware) que las de los desarrolladores del software del

teléfono. Los sistemas de procesamiento por lotes se usan con frecuencia en conjunción con sistemas basados en la Web. Por ejemplo, en una compañía, las solicitudes de gastos de viaje se envían mediante una aplicación Web, aunque se procesa en una aplicación batch para pago mensual.

Para cada tipo de sistema se usan distintas técnicas de ingeniería de software, porque el software tiene características muy diferentes. Por ejemplo, un sistema de control embebido en un automóvil es crítico para la seguridad y se quema en la ROM cuando se instala en el vehículo; por consiguiente, es muy costoso cambiarlo. Tal sistema necesita verificación y validación muy exhaustivas, de tal modo que se minimicen las probabilidades de volver a llamar para revisión a automóviles, después de su venta, para corregir los problemas del software. La interacción del usuario es mínima (o quizás inexistente), por lo que no hay necesidad de usar un proceso de desarrollo que se apoye en el prototipo de interfaz de usuario.

Para un sistema basado en la Web sería adecuado un enfoque basado en el desarrollo y la entrega iterativos, con un sistema de componentes reutilizables. Sin embargo, tal enfoque podría no ser práctico para un sistema de sistemas, donde tienen que definirse por adelantado las especificaciones detalladas de las interacciones del sistema, de modo que cada sistema se desarrolle por separado.

No obstante, existen fundamentos de ingeniería de software que se aplican a todos los tipos de sistema de software:

1. Deben llevarse a cabo usando un proceso de desarrollo administrado y comprendido. La organización que diseña el software necesita planear el proceso de desarrollo, así como tener ideas claras acerca de lo que producirá y el tiempo en que estará completado. Desde luego, se usan diferentes procesos para distintos tipos de software.
2. La confiabilidad y el desempeño son importantes para todos los tipos de sistemas. El software tiene que comportarse como se espera, sin fallas, y cuando se requiera estar disponible. Debe ser seguro en su operación y, tanto como sea posible, también contra ataques externos. El sistema tiene que desempeñarse de manera eficiente y no desperdiciar recursos.
3. Es importante comprender y gestionar la especificación y los requerimientos del software (lo que el software debe hacer). Debe conocerse qué esperan de él los diferentes clientes y usuarios del sistema, y gestionar sus expectativas, para entregar un sistema útil dentro de la fecha y presupuesto.
4. Tiene que usar de manera tan efectiva como sea posible los recursos existentes. Esto significa que, donde sea adecuado, hay que reutilizar el software que se haya desarrollado, en vez de diseñar uno nuevo.

Estas nociones fundamentales sobre proceso, confiabilidad, requerimientos, gestión y reutilización, son temas importantes de este libro. Diferentes métodos los reflejan de formas diversas, pero subyacen en todo el desarrollo de software profesional.

Hay que destacar que estos fundamentos no cubren la implementación ni la programación. En este libro no se estudian técnicas específicas de programación, ya que ellas varían drásticamente de un tipo de sistema a otro. Por ejemplo, un lenguaje de guiones (scripts), como Ruby, sirve para programación de sistemas basados en la Web, aunque sería totalmente inadecuado para ingeniería de sistemas embebidos.

1.1.3 Ingeniería de software y la Web

El desarrollo de la World Wide Web tuvo un profundo efecto en todas nuestras vidas. En un inicio, la Web fue básicamente un almacén de información universal accesible que tuvo escaso efecto sobre los sistemas de software. Dichos sistemas corrían en computadoras locales y eran sólo accesibles desde el interior de una organización. Alrededor del año 2000, la Web comenzó a evolucionar, y a los navegadores se les agregaron cada vez más funcionalidades. Esto significó que los sistemas basados en la Web podían desarrollarse donde se tuviera acceso a dichos sistemas usando un navegador Web, en lugar de una interfaz de usuario de propósito específico. Esta situación condujo al desarrollo de una gran variedad de nuevos productos de sistemas que entregaban servicios innovadores, a los cuales se ingresaba desde la Web. A menudo los financiaban los anuncios publicitarios que se desplegaban en la pantalla del usuario y no requerían del pago directo de los usuarios.

Así como estos productos de sistemas, el desarrollo de navegadores Web que corrieran pequeños programas y realizaran cierto procesamiento local condujo a una evolución en los negocios y el software organizacional. En lugar de elaborar software e implementarlo en las PC de los usuarios, el software se implementaba en un servidor Web. Este avance hizo mucho más barato cambiar y actualizar el software, pues no había necesidad de instalar el software en cada PC. También redujo costos, ya que el desarrollo de interfaces de usuario es bastante caro. En consecuencia, dondequiera que fuera posible hacerlo, muchos negocios se mudaron a la interacción basada en la Web con sistemas de software de la compañía.

La siguiente etapa en el desarrollo de los sistemas basados en la Web fue la noción de los servicios Web. Estos últimos son componentes de software que entregan funcionalidad específica y útil, y a los que se accede desde la Web. Las aplicaciones se construyen al integrar dichos servicios Web que ofrecen diferentes compañías. En principio, esta vinculación suele ser dinámica, de modo que se utilice una aplicación cada vez que se ejecutan diferentes servicios Web. En el capítulo 19 se analiza este acercamiento al desarrollo del software.

En los últimos años se desarrolló la noción de “software como servicio”. Se propuso que el software no correría usualmente en computadoras locales, sino en “nubes de cómputo” a las que se accede a través de Internet. Si usted utiliza un servicio como el correo basado en la Web, usa un sistema basado en nube. Una nube de computación es un enorme número de sistemas de cómputo vinculados que comparten muchos usuarios. Éstos no compran software, sino que pagan según el tiempo de software que se utiliza, o también se les otorga acceso gratuito a cambio de ver anuncios publicitarios que se despliegan en sus pantallas.

Por consiguiente, la llegada de la Web condujo a un significativo cambio en la forma en que se organiza el software empresarial. Antes de la Web, las aplicaciones empresariales eran básicamente monolíticas, los programas corrían en computadoras individuales o en grupos de computadoras. Las comunicaciones eran locales dentro de una organización. Ahora el software está ampliamente distribuido, en ocasiones a lo largo del mundo. Las aplicaciones empresariales no se programan desde cero, sino que requieren la reutilización extensiva de componentes y programas.

En efecto, este cambio radical en la organización del software tuvo que conducir a modificaciones en las formas en que los sistemas basados en la Web se someten a ingeniería. Por ejemplo:

1. La reutilización de software se ha convertido en el enfoque dominante para construir sistemas basados en la Web. Cuando se construyen tales sistemas, uno piensa en cómo ensamblarlos a partir de componentes y sistemas de software preexistentes.

2. Ahora se reconoce en general que no es práctico especificar por adelantado todos los requerimientos para tales sistemas. Los sistemas basados en la Web deben desarrollarse y entregarse de manera progresiva.
3. Las interfaces de usuario están restringidas por las capacidades de los navegadores Web. Aunque tecnologías como AJAX (Holdener, 2008) significan que es posible crear valiosas interfaces dentro de un navegador Web, dichas tecnologías aún son difíciles de emplear. Se usan más comúnmente los formatos Web con escritura de guiones local. Las interfaces de aplicación en sistemas basados en la Web con frecuencia son más deficientes que las interfaces de usuario específicamente diseñadas en productos de sistema PC.

Las ideas fundamentales de la ingeniería de software, discutidas en la sección anterior, se aplican en el software basado en la Web de la misma forma que en otros tipos de sistemas de software. En el siglo XX, la experiencia obtenida con el desarrollo de grandes sistemas todavía es relevante para el software basado en la Web.

1.2 Ética en la ingeniería de software

Como otras disciplinas de ingeniería, la ingeniería de software se realiza dentro de un marco social y legal que limita la libertad de la gente que trabaja en dicha área. Como ingeniero de software, usted debe aceptar que su labor implica responsabilidades mayores que la simple aplicación de habilidades técnicas. También debe comportarse de forma ética y moralmente responsable para ser respetado como un ingeniero profesional.

No sobra decir que debe mantener estándares normales de honestidad e integridad. No debe usar sus habilidades y experiencia para comportarse de forma deshonesto o de un modo que desacredite la profesión de ingeniería de software. Sin embargo, existen áreas donde los estándares de comportamiento aceptable no están acotados por la legislación, sino por la noción más difusa de responsabilidad profesional. Algunas de ellas son:

1. *Confidencialidad* Por lo general, debe respetar la confidencialidad de sus empleadores o clientes sin importar si se firmó o no un acuerdo formal sobre la misma.
2. *Competencia* No debe desvirtuar su nivel de competencia. Es decir, no hay que aceptar de manera intencional trabajo que esté fuera de su competencia.
3. *Derechos de propiedad intelectual* Tiene que conocer las leyes locales que rigen el uso de la propiedad intelectual, como las patentes y el *copyright*. Debe ser cuidadoso para garantizar que se protege la propiedad intelectual de empleadores y clientes.
4. *Mal uso de computadoras* No debe emplear sus habilidades técnicas para usar incorrectamente las computadoras de otros individuos. El mal uso de computadoras varía desde lo relativamente trivial (esto es, distraerse con los juegos de la PC del compañero) hasta lo extremadamente serio (diseminación de virus u otro malware).

Código de ética y práctica profesional de la ingeniería de software

ACM/IEEE-CS Fuerza de trabajo conjunta acerca de ética y prácticas profesionales de la ingeniería de software

PREÁMBULO

La versión corta del código resume las aspiraciones a un alto nivel de abstracción; las cláusulas que se incluyen en la versión completa dan ejemplos y detalles de cómo dichas aspiraciones cambian la forma en que actuamos como profesionales de la ingeniería de software. Sin las aspiraciones, los detalles pueden volverse legalistas y tediosos; mientras que sin los detalles, las aspiraciones suelen volverse muy resonantes pero vacías; en conjunto, aspiraciones y detalles forman un código cohesivo.

Los ingenieros de software deben comprometerse a hacer del análisis, la especificación, el diseño, el desarrollo, la prueba y el mantenimiento del software, una profesión benéfica y respetada. De acuerdo con su compromiso con la salud, la seguridad y el bienestar del público, los ingenieros de software tienen que adherirse a los ocho principios siguientes:

1. PÚBLICO: Los ingenieros de software deben actuar consecuentemente con el interés del público.
2. CLIENTE Y EMPLEADOR: Los ingenieros de software tienen que comportarse de tal forma que fomente el mejor interés para su cliente y empleador, en coherencia con el interés público.
3. PRODUCTO: Los ingenieros de software deben garantizar que sus productos y modificaciones relacionadas satisfagan los estándares profesionales más altos posibles.
4. JUICIO: Los ingenieros de software tienen que mantener integridad e independencia en su juicio profesional.
5. GESTIÓN: Los administradores y líderes en la ingeniería de software deben suscribir y promover un enfoque ético a la gestión del desarrollo y el mantenimiento del software.
6. PROFESIÓN: Los ingenieros de software tienen que fomentar la integridad y la reputación de la profesión consecuente con el interés público.
7. COLEGAS: Los ingenieros de software deben ser justos con sus colegas y apoyarlos.
8. UNO MISMO: Los ingenieros de software tienen que intervenir en el aprendizaje para toda la vida, en cuanto a la práctica de su profesión, y promover un enfoque ético.

Figura 1.3 El código de ética ACM/IEEE
(© IEEE/ACM, 1999)

Las sociedades e instituciones profesionales tienen un importante papel que desempeñar en el establecimiento de estándares éticos. Organizaciones como la ACM, el instituto de ingenieros eléctricos y electrónicos (IEEE) y la British Computer Society publican un código de conducta profesional o código de ética. Los integrantes de tales organizaciones se comprometen a seguir dicho código cuando firman al afiliarse. Estos códigos de conducta se preocupan en general por el comportamiento ético fundamental.

Las asociaciones profesionales, sobre todo la ACM y el IEEE, han cooperado para elaborar conjuntamente un código de ética y práctica profesionales. Este código existe tanto de manera simplificada (figura 1.3) como pormenorizada (Gotterbarn *et al.*, 1999) que agrega detalle y sustancia a la versión más corta. Los fundamentos detrás de este código se resumen en los primeros dos párrafos de la forma pormenorizada:

Las computadoras tienen una función central y creciente en el comercio, la industria, el gobierno, la medicina, la educación, el entretenimiento y la sociedad en general. Los ingenieros de software son quienes contribuyen, mediante la participación directa o con la enseñanza, al análisis, la especificación, el diseño, el desarrollo, la certificación, el mantenimiento y la prueba de los sistemas de software.

Debido a su función en el desarrollo de los sistemas de software, los ingenieros de software tienen oportunidades significativas para hacer lo correcto o causar daño, para permitir que otros hagan lo correcto o causen daño, o para influir en otros para hacer lo correcto o causar daño. Para garantizar, tanto como sea posible, que sus esfuerzos serán usados correctamente, los ingenieros de software deben comprometerse a hacer de la ingeniería de software una profesión benéfica y respetada. En concordancia con dicho compromiso, los ingenieros de software tienen que adherirse al siguiente Código de Ética y Práctica Profesional.

El código contiene ocho principios relacionados con el comportamiento y las decisiones tomadas por ingenieros de software profesionales, incluidos practicantes, educadores, administradores, supervisores y políticos, así como por aprendices y estudiantes de la profesión. Los principios identifican las relaciones éticamente responsables en las que participan individuos, grupos y organizaciones, así como las obligaciones principales dentro de estas relaciones. Las cláusulas de cada principio son ilustraciones de algunas de las obligaciones incluidas en dichas relaciones. Tales obligaciones se fundamentan en el sentido humano del ingeniero de software, en el cuidado especial que se debe a las personas afectadas por el trabajo de los ingenieros de software, y los elementos únicos de la práctica de la ingeniería de software. El código las formula como obligaciones de quienquiera que afirme o aspire a ser ingeniero de software.

En cualquier situación donde distintos individuos tengan diferentes visiones y objetivos, es probable que usted enfrente dilemas éticos. Por ejemplo, si está en desacuerdo, en principio, con las políticas de los ejecutivos de más alto nivel en la compañía, ¿cómo reaccionaría? Claramente, esto depende de cada individuo y de la naturaleza de la discrepancia. ¿Es mejor argumentar un caso para su posición desde el interior de la organización o renunciar en principio? Si siente que existen problemas con un proyecto de software, ¿cuándo los reporta a la administración? Si los discute mientras apenas son un indicio, puede estar exagerando su reacción ante una situación; y si los deja para más tarde, quizá sea ya imposible resolver las dificultades.

Estos dilemas éticos los enfrentamos todos en la vida profesional y, por fortuna, en la mayoría de los casos son relativamente menores o pueden resolverse sin demasiada dificultad. En caso de que no puedan solucionarse, el ingeniero afronta, tal vez, otro problema. La acción basada en los principios quizá sea renunciar al empleo, aunque esta decisión bien podría afectar a otros, como a su pareja o a sus hijos.

Una situación muy difícil para los ingenieros profesionales surge cuando su empleador actúa sin ética. Es decir, una compañía es responsable del desarrollo de un sistema crítico de seguridad y, debido a presión del tiempo, falsifica los registros de validación de seguridad. ¿Es responsabilidad del ingeniero mantener la confidencialidad o alertar al cliente o manifestar, de alguna forma, que el sistema entregado quizá sea inseguro?

El problema aquí es que no hay absolutos cuando se trata de seguridad. Aunque el sistema pueda no estar validado de acuerdo con criterios predefinidos, dichos criterios quizá sean demasiado estrictos. En realidad el sistema operará con seguridad a lo largo de su vida. También está el caso de que, aun cuando se valide de manera adecuada, el sistema falle y cause un accidente. La detección oportuna de los problemas puede resultar lesiva para el empleador y otros trabajadores; el fracaso por revelar los problemas podría ser dañino para otros.

El lector debe formar su propio criterio en estos asuntos. Aquí, la posición ética adecuada depende por completo de las percepciones de los individuos que están implicados. En este caso, el potencial de daño, el alcance del mismo y las personas afectadas deben influir en la decisión. Si el escenario es muy peligroso, estaría justificado anunciarlo a través de la prensa nacional (por ejemplo). Sin embargo, siempre hay que tratar de resolver la situación sin dejar de respetar los derechos de su empleador.

Otro conflicto ético es la participación en el desarrollo de sistemas militares y nucleares. Al respecto, algunas personas se sienten muy afectadas por estos temas y evitan participar en el desarrollo de algún sistema asociado con los sistemas militares. Otras más trabajarán en los sistemas militares, pero no en los de armamento. Incluso otras sentirán que la seguridad nacional es un principio fundamental y no tienen objeciones éticas para trabajar en sistemas de armamento.

En tal situación es importante que tanto empleadores como empleados dejen en claro con antelación sus percepciones o puntos de vista. Cuando una organización participa en trabajo militar o nuclear, debe contar con la capacidad de especificar que los empleados tienen la voluntad de aceptar cualquier trabajo asignado. De igual forma, si un empleado toma la responsabilidad y deja en claro que no quiere trabajar en tales sistemas, los empleadores no tendrán que presionarlo para que éste lo haga más tarde.

El área general de la ética y la responsabilidad profesional se vuelven más importantes conforme los sistemas intensivos en software prevalecen en cada vez más cuestiones del trabajo y la vida cotidiana. Puede considerarse desde un punto de vista filosófico, donde se tomen en cuenta los principios básicos de la ética y se analice la ética de la ingeniería de software en relación con dichos principios básicos. Éste es el enfoque que toma Laudon (1995) y, en menor medida, Huff y Martin (1995). El texto de Johnson sobre ética computacional (2001) también trata el tema desde una perspectiva filosófica.

Sin embargo, este enfoque filosófico resulta muy abstracto y difícil de relacionar con la experiencia cotidiana. Es preferible el enfoque más concreto plasmado en los códigos de conducta y práctica. Se considera que la ética se analiza mejor en un contexto de ingeniería de software y no como un tema por derecho propio. Por lo tanto, en este libro no se presentan, donde es adecuado, discusiones éticas abstractas, sino que se incluyen ejemplos en los ejercicios que son el punto de partida para una discusión grupal sobre conflictos éticos.

1.3 Estudios de caso

Para ilustrar los conceptos de la ingeniería de software, a lo largo del libro se utilizan ejemplos de tres tipos de sistemas diferentes. La razón de no usar un solo estudio de caso obedece a que uno de los mensajes clave de este libro es que la práctica de la ingeniería de software depende del tipo de sistemas a producir. Por consiguiente, se elegirá un ejemplo adecuado cuando se estudien conceptos como seguridad y confiabilidad, modo de sistema, reutilización, etcétera.

Los tres tipos de sistemas que se usan como estudios de caso son:

1. *Un sistema embebido* Se trata de un sistema donde el software controla un dispositivo de hardware y está *embebido* en dicho dispositivo. Los conflictos en los sistemas *embebidos* incluyen por lo general tamaño físico, capacidad de reacción,

administración de la energía, etcétera. El ejemplo de un sistema *embebido* utilizado es un sistema de software para controlar un dispositivo médico.

2. *Un sistema de información* Es un sistema cuyo principal propósito es gestionar y dar acceso a una base de datos de información. Los conflictos en los sistemas de información incluyen seguridad, usabilidad, privacidad y mantenimiento de la integridad de los datos. Un sistema de registros médicos se utiliza como ejemplo de un sistema de información.
3. *Un sistema de adquisición de datos basado en sensores* Se trata de un sistema cuyo principal objetivo es recolectar datos de un conjunto de sensores y procesar esos datos de alguna forma. Los requerimientos clave de tales sistemas son fiabilidad, incluso en condiciones de ambientes hostiles, y capacidad de mantenimiento. Una estación meteorológica a campo abierto es el ejemplo que se usa como sistema de adquisición de datos.

En este capítulo se introduce cada uno de dichos sistemas, y sobre todos ellos hay más información disponible en la Web.

1.3.1 Sistema de control para una bomba de insulina

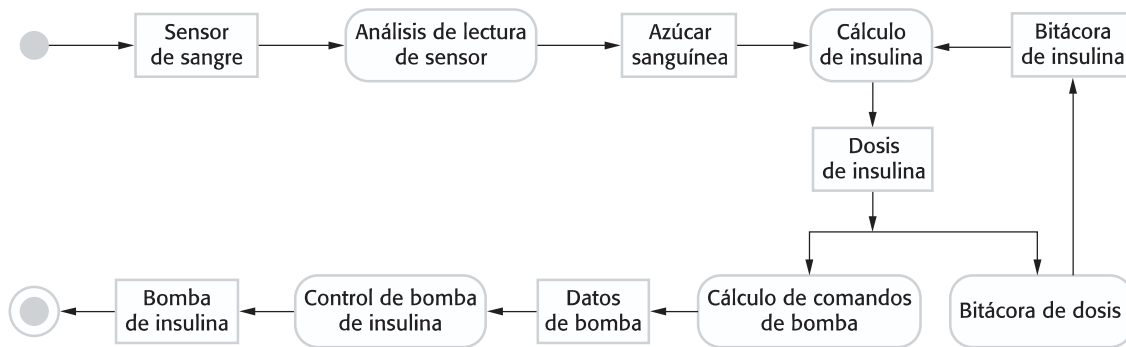
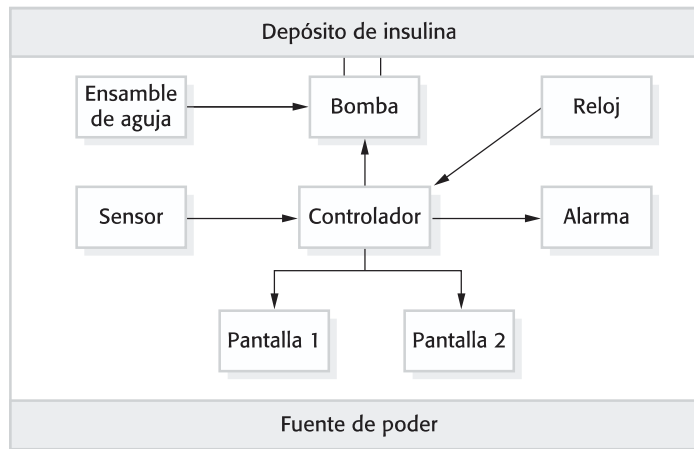
Una bomba de insulina es un sistema médico que simula la función del páncreas (un órgano interno). El software que controla este sistema es un sistema embebido, que recopila información de un sensor y controla una bomba que entrega al usuario una dosis regulada de insulina.

Las personas que sufren de diabetes usan el sistema. La diabetes es relativamente una condición común, donde el páncreas humano es incapaz de producir suficientes cantidades de una hormona llamada insulina. La insulina metaboliza la glucosa (azúcar) en la sangre. El tratamiento convencional de la diabetes incluye inyecciones regulares de insulina genéticamente manipulada. Los diabéticos calculan sus niveles de azúcar en la sangre usando un medidor externo y, luego, ajustan la dosis de insulina que deben inyectarse.

El problema con este tratamiento es que el nivel de insulina requerido no depende sólo del nivel de glucosa en la sangre, sino también del tiempo desde la última inyección de insulina. Esto podría conducir a niveles muy bajos de glucosa sanguínea (si hay mucha insulina) o niveles muy altos de azúcar sanguínea (si hay muy poca insulina). La baja en glucosa sanguínea es, a corto plazo, una condición más seria que puede resultar en mal funcionamiento temporal del cerebro y, finalmente, en inconsciencia y muerte. Y por otro lado, a largo plazo los continuos niveles elevados de glucosa en la sangre ocasionan daño ocular, renal y problemas cardíacos.

Los avances recientes en el desarrollo de sensores miniaturizados significan que ahora es posible desarrollar sistemas automatizados de suministro de insulina. Dichos sistemas monitorizan los niveles de azúcar en la sangre y, cuando se requiere, administran una dosis adecuada de insulina. Los sistemas de entrega de insulina como éste ya existen para el tratamiento de pacientes hospitalarios. En el futuro, muchos diabéticos tendrán tales sistemas permanentemente unidos a sus cuerpos.

Un sistema de suministro de insulina controlado por software puede funcionar al usar un microsensor embebido en el paciente, con la finalidad de medir ciertos parámetros sanguíneos que sean proporcionales al nivel de azúcar. Luego, esto se envía al controlador de la bomba, el cual calcula el nivel de azúcar y la cantidad de insulina que se nece-

Figura 1.4 Hardware de bomba de insulina**Figura 1.5** Modelo de actividad de la bomba de insulina

sita. Entonces envía señales a una bomba miniaturizada para administrar la insulina vía una aguja permanentemente unida.

La figura 1.4 muestra los componentes de hardware y la organización de la bomba de insulina. Para entender los ejemplos, todo lo que necesita saber es que el sensor de sangre mide la conductividad eléctrica de la sangre bajo diferentes condiciones y que dichos valores podrían relacionarse con el nivel de azúcar en la sangre. La bomba de insulina entrega una unidad de insulina en respuesta a un solo pulso de un controlador. Por lo tanto, para entregar 10 unidades de insulina, el controlador envía 10 pulsos a la bomba. La figura 1.5 es un modelo de actividad UML que ilustra cómo el software transforma una entrada de nivel de azúcar en la sangre, con una secuencia de comandos que impulsan la bomba de insulina.

Claramente, éste es un sistema crítico de seguridad. Si la bomba no opera o no lo hace de manera correcta, entonces la salud del usuario estaría en grave riesgo o éste caería en estado de coma debido a que sus niveles de azúcar en la sangre son muy altos o muy bajos. En consecuencia, hay dos requerimientos esenciales de alto nivel que debe satisfacer este sistema:

1. El sistema tiene que estar disponible para entregar insulina cuando se requiera.
2. El sistema requiere funcionar de manera confiable y entregar la cantidad correcta de insulina, para contrarrestar el nivel actual de azúcar en la sangre.

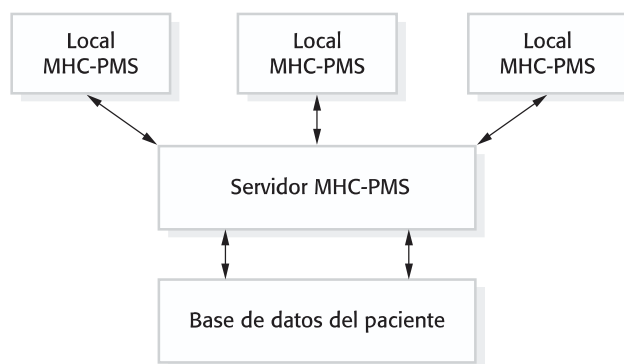


Figura 1.6 Organización del MHC-PMS

Por consiguiente, el sistema debe diseñarse e implementarse para garantizar que siempre satisfaga dichos requerimientos. En capítulos posteriores se estudian requerimientos más detallados y se discute acerca de cómo probar que el sistema sea seguro.

1.3.2 Sistema de información de pacientes para atención a la salud mental

Un sistema de información de pacientes para apoyar la atención a la salud mental es un sistema de información médica, que administra la información de pacientes que sufren problemas de salud mental y los tratamientos que reciben. La mayoría de los pacientes con problemas de salud mental no requieren tratamiento hospitalario dedicado, pero sí asistir regularmente a clínicas especializadas donde se reúnen con un médico que tiene conocimiento detallado de sus problemas. Para facilitar la asistencia de los pacientes, dichas clínicas no sólo funcionan en los hospitales sino también en consultorios médicos locales o centros comunitarios.

El MHC-PMS (sistema de administración de pacientes-atención a la salud mental) es un sistema de información destinado para usarse en clínicas. Utiliza una base de datos centralizada con información de los pacientes, aunque también se diseñó para operarse desde una PC, de modo que se puede acceder a ella y usarse desde sitios sin conectividad de red segura. Cuando los sistemas locales tienen acceso seguro a red, emplean la información de los pacientes en la base de datos, pero además son capaces de descargar y usar copias locales de registros de pacientes cuando los sistemas están desconectados. El sistema no es un sistema de registros médicos completo, por lo que no conserva información acerca de otras condiciones médicas. Sin embargo, interactúa e intercambia datos con otros sistemas de información clínica. La figura 1.6 ilustra la organización del MHC-PMS.

El MHC-PMS tiene dos metas globales:

1. Generar información de gestión que permita a los administradores de servicios de salud valorar el desempeño contra objetivos locales y de gobierno.
2. Proporcionar al personal médico información oportuna para apoyar el tratamiento de los pacientes.

La naturaleza de los problemas de salud mental es tal que los pacientes se hallan con frecuencia desorganizados y suelen faltar a sus citas, deliberada o accidentalmente, perder recetas y medicamentos, olvidar instrucciones y realizar demandas irracionales al personal médico. Pueden llegar a las clínicas de manera inesperada. En muy pocos casos, son un riesgo para sí mismos o para otros individuos. Regularmente pueden cambiar de dirección o no tener casa a corto o largo plazo. Cuando los pacientes son peligrosos, quizá deban “internarse”: confinarse en un hospital seguro para tratamiento y observación.

Los usuarios del sistema incluyen personal clínico como médicos, enfermeros y visitadores de salud (enfermeros que visitan a las personas a domicilio para verificar su tratamiento). Los usuarios no médicos incluyen recepcionistas que hacen citas, personal de archivo médico que organiza el sistema de registros, y personal administrativo que redacta informes.

El sistema sirve para registrar información de pacientes (nombre, dirección, edad, pariente más cercano, etcétera), consultas (fecha, médico, impresiones personales del paciente, etcétera), condiciones y tratamientos. Los informes se elaboran a intervalos regulares para el personal médico y los administradores de la autoridad sanitaria. Por lo general, los reportes para el personal médico se enfocan en la información individual de pacientes, mientras que los reportes de la administración son anónimos y se interesan por las condiciones, costos de tratamiento, etcétera.

Las características clave del sistema son:

1. *Administración de atención individual* Los médicos de atención primaria crean registros para pacientes, editan la información en el sistema, ven el historial del paciente, etcétera. El sistema soporta resúmenes de datos para que los médicos que no se reunieron con anterioridad con el paciente se enteren rápidamente de los problemas y tratamientos clave que se prescribieron.
2. *Monitorización del paciente* El sistema monitoriza regularmente los registros de los pacientes que están involucrados en tratamiento y emite advertencias cuando se detectan posibles dificultades. En consecuencia, si un paciente no ha visto a un médico durante cierto tiempo, puede emitirse una advertencia. Uno de los elementos más importantes del sistema de monitorización es seguir la pista de los pacientes que fueron internados y garantizar que las verificaciones requeridas legalmente se lleven a cabo en el tiempo correcto.
3. *Informes administrativos* El sistema genera mensualmente informes administrativos que muestran el número de pacientes tratados en cada clínica, la cantidad de pacientes que ingresaron y salieron del sistema de salud, el total de pacientes internados, los medicamentos prescritos y sus costos, etcétera.

Dos leyes diferentes afectan al sistema. Se trata de leyes de protección de datos que rigen la confidencialidad de la información personal, y las leyes de salud mental, que establecen la detención obligatoria de los pacientes considerados como un peligro para sí mismos o para otros. La salud mental es única en este aspecto, pues es la única especialidad médica que puede recomendar la detención de pacientes contra la voluntad de éstos, lo cual está sujeto a protecciones legislativas muy estrictas. Una de las metas del MHC-PMS es asegurar que el personal siempre actúe en concordancia con la ley y que sus decisiones, si es necesario, se registren para revisión judicial.

Como en todos los sistemas médicos, la privacidad es un requerimiento de sistema crítico. Es básico que la información de los pacientes sea confidencial y nunca se revele a nadie



Figura 1.7 El entorno de la estación meteorológica

más, aparte del personal médico autorizado y los mismos pacientes. El MHC-PMS también es un sistema crítico de seguridad. Algunas patologías mentales hacen que los pacientes se vuelvan suicidas o un peligro para otros individuos. Siempre que sea posible, el sistema debe advertir al personal médico acerca de pacientes potencialmente suicidas o peligrosos.

El diseño global del sistema debe considerar requerimientos de privacidad y seguridad. El sistema tiene que estar disponible cuando se necesite, de otro modo la seguridad estaría comprometida y sería imposible prescribir a los pacientes el medicamento correcto. Aquí existe un conflicto potencial: la privacidad es más fácil de mantener cuando existe sólo una copia de los datos del sistema. Sin embargo, para garantizar la disponibilidad en el caso de fallas del servidor o desconexión de una red, hay que conservar varias copias de los datos. En capítulos posteriores se analizan las preferencias temporales entre tales requerimientos.

1.3.3 Estación meteorológica a campo abierto

Para ayudar a monitorizar el cambio climático y mejorar la exactitud de las predicciones meteorológicas en áreas remotas, el gobierno de un país con grandes áreas de campo abierto decidió instalar varios cientos de estaciones meteorológicas en dichas áreas. Las estaciones meteorológicas recopilan datos de un conjunto de instrumentos que miden temperatura y presión, luz solar, lluvia, y rapidez y dirección del viento.

Las estaciones meteorológicas a campo abierto son parte de un sistema más grande (figura 1.7), que es un sistema de información meteorológica que recolecta datos de estaciones meteorológicas y los pone a disposición de otros sistemas para su procesamiento. Los sistemas en la figura 1.7 son:

1. *El sistema de estación meteorológica* Es responsable de recolectar datos meteorológicos, realizar cierto procesamiento de datos inicial y transmitirlo al sistema de gestión de datos.
2. *El sistema de gestión y archivado de datos* Recolecta los datos de todas las estaciones meteorológicas a campo abierto, realiza procesamiento y análisis de datos, y los archiva de forma que los puedan recuperar otros sistemas, como los sistemas de predicción meteorológica.
3. *El sistema de mantenimiento de estación* Se comunica por satélite con todas las estaciones meteorológicas a campo abierto, para monitorizar el estado de dichos sistemas y dar reportes sobre problemas. Puede actualizar el software embebido en dichos sistemas. En caso de problemas del sistema, también sirve para controlar de manera remota un sistema meteorológico a campo abierto.

En la figura 1.7 se usó el símbolo de paquete UML para indicar que cada sistema es una colección de componentes, y se identificaron los sistemas separados usando el estereotipo UML «sistema». Las asociaciones entre los paquetes indican que ahí existe un intercambio de información pero, en esta etapa, no hay necesidad de definirlos con más detalle.

Cada estación meteorológica incluye algunos instrumentos que miden parámetros climatológicos como rapidez y dirección del viento, temperaturas del terreno y aire, presión barométrica y lluvia durante un periodo de 24 horas. Cada uno de dichos instrumentos está controlado por un sistema de software que toma periódicamente lecturas de parámetros y gestiona los datos recolectados desde los instrumentos.

El sistema de estación meteorológica opera mediante la recolección de observaciones meteorológicas a intervalos frecuentes; por ejemplo, las temperaturas se miden cada minuto. Sin embargo, puesto que el ancho de banda del satélite es relativamente estrecho, la estación meteorológica realiza cierto procesamiento local y concentración de los datos. Luego, transmite los datos concentrados cuando los solicita el sistema de adquisición de datos. Pero si, por cualquier razón, es imposible realizar una conexión, entonces la estación meteorológica mantiene los datos localmente hasta que se reanude la comunicación.

Cada estación meteorológica es alimentada por baterías y debe estar completamente autocontenida: no hay fuentes de energía externas o cables de red disponibles. Todas las comunicaciones son a través de un vínculo satelital de rapidez relativamente baja, y la estación meteorológica debe incluir algún mecanismo (solar o eólico) para cargar sus baterías. Puesto que se despliegan en áreas abiertas, están expuestas a severas condiciones ambientales y los animales llegan a dañarlas. Por lo tanto, el software de la estación no sólo se encarga de la adquisición de datos. También debe:

1. Monitorizar los instrumentos, la energía y el hardware de comunicación, y reportar los fallas al sistema de administración.
2. Administrar la energía del sistema, garantizar que las baterías estén cargadas siempre que las condiciones ambientales lo permitan; así como desconectar los generadores ante condiciones meteorológicas potencialmente adversas, como viento fuerte.
3. Permitir la reconfiguración dinámica donde partes del software se sustituyan con nuevas versiones, y los instrumentos de respaldo se enciendan en el sistema en caso de falla de éste.

Puesto que las estaciones meteorológicas deben estar autocontenidas y sin vigilancia, esto significa que el software instalado es complejo, aun cuando la funcionalidad de adquisición de datos sea bastante simple.

PUNTOS CLAVE

- La ingeniería de software es una disciplina de ingeniería que se interesa por todos los aspectos de la producción de software.
- El software no es sólo un programa o programas, sino que también incluye documentación. Los atributos esenciales de los productos de software son mantenimiento, confiabilidad, seguridad, eficiencia y aceptabilidad.
- El proceso de software incluye todas las actividades que intervienen en el desarrollo de software. Las actividades de alto nivel de especificación, desarrollo, validación y evolución son parte de todos los procesos de software.
- Las nociones fundamentales de la ingeniería de software son universalmente aplicables a todos los tipos de desarrollo de sistema. Dichos fundamentos incluyen procesos, confiabilidad, seguridad, requerimientos y reutilización de software.
- Existen muchos tipos diferentes de sistemas y cada uno requiere para su desarrollo de herramientas y técnicas adecuadas de ingeniería de software. Existen pocas, si es que hay alguna, técnicas específicas de diseño e implementación que son aplicables a todos los tipos de sistemas.
- Las ideas fundamentales de la ingeniería de software son aplicables a todos los tipos de sistemas de software. Dichos fundamentos incluyen procesos de administración de software, confiabilidad y seguridad del software, ingeniería de requerimientos y reutilización de software.
- Los ingenieros de software tienen responsabilidades con la profesión de ingeniería y la sociedad. No deben preocuparse únicamente por temas técnicos.
- Las sociedades profesionales publican códigos de conducta que establecen los estándares de comportamiento esperados de sus miembros.

LECTURAS SUGERIDAS

“No silver bullet: Essence and accidents of software engineering”. A pesar de su fecha de publicación, este artículo es una buena introducción general a los problemas de la ingeniería de software. El mensaje esencial del artículo no ha cambiado. (F. P. Brooks, *IEEE Computer*, **20** (4), abril 1987.) <http://doi.ieeecomputersociety.org/10.1109/MC.1987.1663532>.

“Software engineering code of ethics is approved”. Un artículo que analiza a fondo los antecedentes para el desarrollo del Código de ética ACM/IEEE y que incluye las formas corta y larga del código. (*Comm. ACM*, D. Gotterbarn, K. Miller, y S. Rogerson, octubre 1999.) <http://portal.acm.org/citation.cfm?doid=317665.317682>.

Professional Issues in Software Engineering. Éste es un excelente libro que examina conflictos legales y profesionales, así como éticos. El autor prefiere su enfoque práctico a textos más teóricos acerca de la ética. (F. Bott, A. Coleman, J. Eaton y D. Rowland, 3ª. edición, 2000, Taylor and Francis.)

IEEE Software, March/April 2002. Éste es un número especial de la revista dedicado al desarrollo de software basado en la Web. Esta área ha cambiado muy rápidamente, de modo que algunos artículos son un poco arcaicos, pero la mayoría todavía son relevantes. (*IEEE Software*, **19** (2), 2002.) <http://www2.computer.org/portal/web/software>.

“A View of 20th and 21st Century Software Engineering”. Un vistazo atrás y adelante a la ingeniería de software de uno de los primeros y más distinguidos ingenieros de software. Barry Boehm identifica principios atemporales de la ingeniería de software, pero también sugiere que algunas prácticas de uso común son obsoletas. (B. Boehm, *Proc. 28th Software Engineering Conf.*, Shanghai, 2006.) <http://doi.ieeecomputersociety.org/10.1145/1134285.1134288>.

“Software Engineering Ethics”. Número especial de *IEEE Computer*, con algunos artículos acerca del tema. (*IEEE Computer*, **42** (6), junio 2009.)

EJERCICIOS

- 1.1. Explique por qué el software profesional no sólo son programas que se desarrollan para un cliente.
- 1.2. ¿Cuál es la principal diferencia entre desarrollo de productos de software genéricos y desarrollo de software personalizado? ¿Qué significa esto en la práctica para los usuarios de productos de software genérico?
- 1.3. ¿Cuáles son los cuatro atributos importantes que debe tener todo software profesional? Sugiera otros cuatro atributos que en ocasiones sean significativos.
- 1.4. Además de los retos de la heterogeneidad, cambio empresarial y social, y confianza y seguridad, identifique otros problemas y retos que sea probable que enfrente la ingeniería de software en el siglo XXI. (Sugerencia: piense en el ambiente).
- 1.5. Con base en su conocimiento de algunos tipos de aplicación estudiados en la sección 1.1.2, explique, con ejemplos, por qué diferentes tipos de aplicación requieren técnicas especializadas de ingeniería de software, para apoyar su diseño y desarrollo.
- 1.6. Explique por qué existen ideas fundamentales de la ingeniería de software que se aplican a todos los tipos de sistemas de software.
- 1.7. Explique cómo el uso universal de la Web cambió los sistemas de software.
- 1.8. Analice el hecho de si los ingenieros profesionales deben ser certificados en la misma forma que los médicos o abogados.
- 1.9. Para cada una de las cláusulas del Código de ética ACM/IEEE que se muestra en la figura 1.3, sugiera un ejemplo adecuado que ilustre dicha cláusula.
- 1.10. Para ayudar a contrarrestar el terrorismo, muchos países planean o desarrollaron sistemas de cómputo que siguen la pista a gran cantidad de sus ciudadanos y sus acciones. Claramente esto tiene implicaciones en cuanto a la privacidad. Discuta la ética de trabajar en el desarrollo de este tipo de sistema.

REFERENCIAS

Gotterbarn, D., Miller, K. y Rogerson, S. (1999). Software Engineering Code of Ethics is Approved. *Comm. ACM*, **42** (10), 102–7.

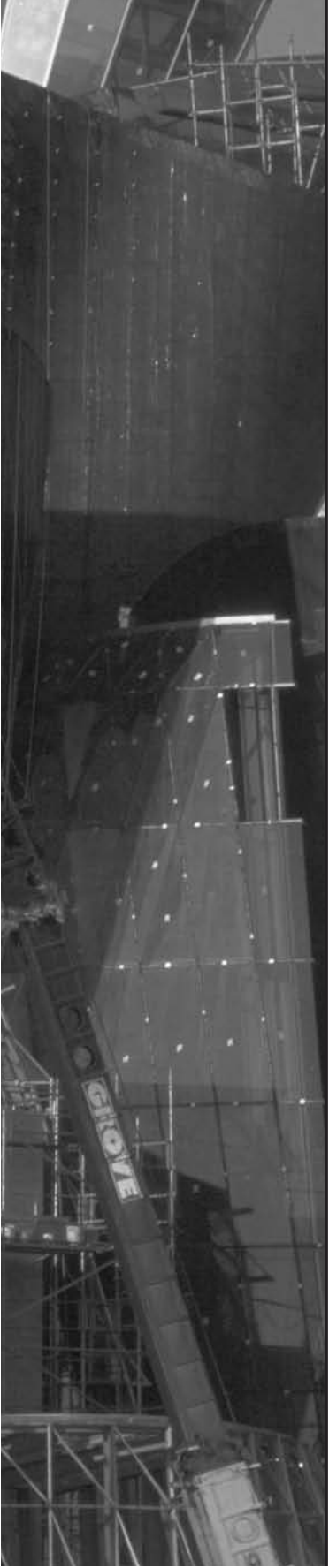
Holdener, A. T. (2008). *Ajax: The Definitive Guide*. Sebastopol, Ca.: O'Reilly and Associates.

Huff, C. y Martin, C. D. (1995). Computing Consequences: A Framework for Teaching Ethical Computing. *Comm. ACM*, **38** (12), 75–84.

Johnson, D. G. (2001). *Computer Ethics*. Englewood Cliffs, NJ: Prentice Hall.

Laudon, K. (1995). Ethical Concepts and Information Technology. *Comm. ACM*, **38** (12), 33–9.

Naur, P. y Randell, B. (1969). Ingeniería de software: Reporte de una conferencia patrocinada por el comité científico de la OTAN, Garmisch, Alemania, 7 a 11 de octubre de 1968.



2

Procesos de software

Objetivos

El objetivo de este capítulo es introducirlo hacia la idea de un proceso de software: un conjunto coherente de actividades para la producción de software. Al estudiar este capítulo:

- comprenderá los conceptos y modelos sobre procesos de software;
- se introducirá en los tres modelos de proceso de software genérico y sabrá cuándo usarlos;
- entenderá las principales actividades del proceso de ingeniería de requerimientos de software, así como del desarrollo, las pruebas y la evolución del software;
- comprenderá por qué deben organizarse los procesos para enfrentar los cambios en los requerimientos y el diseño de software;
- entenderá cómo el Proceso Unificado Racional (Rational Unified Process, RUP) integra buenas prácticas de ingeniería de software para crear procesos de software adaptables.

Contenido

- 2.1 Modelos de proceso de software
- 2.2 Actividades del proceso
- 2.3 Cómo enfrentar el cambio
- 2.4 El Proceso Unificado Racional

Un proceso de software es una serie de actividades relacionadas que conduce a la elaboración de un producto de software. Estas actividades pueden incluir el desarrollo de software desde cero en un lenguaje de programación estándar como Java o C. Sin embargo, las aplicaciones de negocios no se desarrollan precisamente de esta forma. El nuevo software empresarial con frecuencia ahora se desarrolla extendiendo y modificando los sistemas existentes, o configurando e integrando el software comercial o componentes del sistema.

Existen muchos diferentes procesos de software, pero todos deben incluir cuatro actividades que son fundamentales para la ingeniería de software:

1. *Especificación del software* Tienen que definirse tanto la funcionalidad del software como las restricciones de su operación.
2. *Diseño e implementación del software* Debe desarrollarse el software para cumplir con las especificaciones.
3. *Validación del software* Hay que validar el software para asegurarse de que cumple lo que el cliente quiere.
4. *Evolución del software* El software tiene que evolucionar para satisfacer las necesidades cambiantes del cliente.

En cierta forma, tales actividades forman parte de todos los procesos de software. Por supuesto, en la práctica éstas son actividades complejas en sí mismas e incluyen subactividades tales como la validación de requerimientos, el diseño arquitectónico, la prueba de unidad, etcétera. También existen actividades de soporte al proceso, como la documentación y el manejo de la configuración del software.

Cuando los procesos se discuten y describen, por lo general se habla de actividades como especificar un modelo de datos, diseñar una interfaz de usuario, etcétera, así como del orden de dichas actividades. Sin embargo, al igual que las actividades, también las descripciones de los procesos deben incluir:

1. Productos, que son los resultados de una actividad del proceso. Por ejemplo, el resultado de la actividad del diseño arquitectónico es un modelo de la arquitectura de software.
2. Roles, que reflejan las responsabilidades de la gente que interviene en el proceso. Ejemplos de roles: gerente de proyecto, gerente de configuración, programador, etcétera.
3. Precondiciones y postcondiciones, que son declaraciones válidas antes y después de que se realice una actividad del proceso o se cree un producto. Por ejemplo, antes de comenzar el diseño arquitectónico, una precondición es que el cliente haya aprobado todos los requerimientos; después de terminar esta actividad, una postcondición podría ser que se revisen aquellos modelos UML que describen la arquitectura.

Los procesos de software son complejos y, como todos los procesos intelectuales y creativos, se apoyan en personas con capacidad de juzgar y tomar decisiones. No hay un proceso ideal; además, la mayoría de las organizaciones han diseñado sus propios procesos de desarrollo de software. Los procesos han evolucionado para beneficiarse de las capacidades de la gente en una organización y de las características específicas de los

sistemas que se están desarrollando. Para algunos sistemas, como los sistemas críticos, se requiere de un proceso de desarrollo muy estructurado. Para los sistemas empresariales, con requerimientos rápidamente cambiantes, es probable que sea más efectivo un proceso menos formal y flexible.

En ocasiones, los procesos de software se clasifican como dirigidos por un plan (*plan-driven*) o como procesos ágiles. Los procesos dirigidos por un plan son aquellos donde todas las actividades del proceso se planean por anticipado y el avance se mide contra dicho plan. En los procesos ágiles, que se estudiarán en el capítulo 3, la planeación es incremental y es más fácil modificar el proceso para reflejar los requerimientos cambiantes del cliente. Como plantean Boehm y Turner (2003), cada enfoque es adecuado para diferentes tipos de software. Por lo general, uno necesita encontrar un equilibrio entre procesos dirigidos por un plan y procesos ágiles.

Aunque no hay un proceso de software “ideal”, en muchas organizaciones sí existe un ámbito para mejorar el proceso de software. Los procesos quizás incluyan técnicas obsoletas o tal vez no aprovechen las mejores prácticas en la industria de la ingeniería de software. En efecto, muchas organizaciones aún no sacan ventaja de los métodos de la ingeniería de software en su desarrollo de software.

Los procesos de software pueden mejorarse con la estandarización de los procesos, donde se reduce la diversidad en los procesos de software en una organización. Esto conduce a mejorar la comunicación, a reducir el tiempo de capacitación, y a que el soporte de los procesos automatizados sea más económico. La estandarización también representa un primer paso importante tanto en la introducción de nuevos métodos y técnicas de ingeniería de software, como en sus buenas prácticas. En el capítulo 26 se analiza con más detalle la mejora en el proceso de software.

2.1 Modelos de proceso de software

Como se explicó en el capítulo 1, un modelo de proceso de software es una representación simplificada de este proceso. Cada modelo del proceso representa a otro desde una particular perspectiva y, por lo tanto, ofrece sólo información parcial acerca de dicho proceso. Por ejemplo, un modelo de actividad del proceso muestra las actividades y su secuencia, pero quizá sin presentar los roles de las personas que intervienen en esas actividades. En esta sección se introducen algunos modelos de proceso muy generales (en ocasiones llamados “paradigmas de proceso”) y se muestran desde una perspectiva arquitectónica. En otras palabras, se ve el marco (framework) del proceso, pero no los detalles de las actividades específicas.

Tales modelos genéricos no son descripciones definitivas de los procesos de software. Más bien, son abstracciones del proceso que se utilizan para explicar los diferentes enfoques del desarrollo de software. Se pueden considerar marcos del proceso que se extienden y se adaptan para crear procesos más específicos de ingeniería de software.

Los modelos del proceso que se examinan aquí son:

1. *El modelo en cascada (waterfall)* Éste toma las actividades fundamentales del proceso de especificación, desarrollo, validación y evolución y, luego, los representa como fases separadas del proceso, tal como especificación de requerimientos, diseño de software, implementación, pruebas, etcétera.

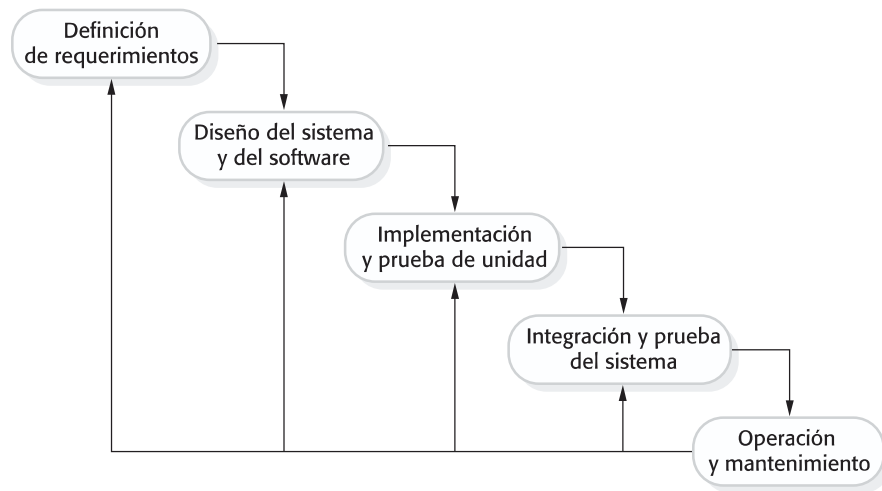


Figura 2.1 El modelo en cascada

2. *Desarrollo incremental* Este enfoque vincula las actividades de especificación, desarrollo y validación. El sistema se desarrolla como una serie de versiones (incrementos), y cada versión añade funcionalidad a la versión anterior.
3. *Ingeniería de software orientada a la reutilización* Este enfoque se basa en la existencia de un número significativo de componentes reutilizables. El proceso de desarrollo del sistema se enfoca en la integración de estos componentes en un sistema, en vez de desarrollarlo desde cero.

Dichos modelos no son mutuamente excluyentes y con frecuencia se usan en conjunto, sobre todo para el desarrollo de grandes sistemas. Para este tipo de sistemas, tiene sentido combinar algunas de las mejores características de los modelos de desarrollo en cascada e incremental. Se necesita contar con información sobre los requerimientos esenciales del sistema para diseñar la arquitectura de software que apoye dichos requerimientos. No puede desarrollarse de manera incremental. Los subsistemas dentro de un sistema más grande se desarrollan usando diferentes enfoques. Partes del sistema que son bien comprendidas pueden especificarse y desarrollarse al utilizar un proceso basado en cascada. Partes del sistema que por adelantado son difíciles de especificar, como la interfaz de usuario, siempre deben desarrollarse con un enfoque incremental.

2.1.1 El modelo en cascada

El primer modelo publicado sobre el proceso de desarrollo de software se derivó a partir de procesos más generales de ingeniería de sistemas (Royce, 1970). Este modelo se ilustra en la figura 2.1. Debido al paso de una fase en cascada a otra, este modelo se conoce como “modelo en cascada” o ciclo de vida del software. El modelo en cascada es un ejemplo de un proceso dirigido por un plan; en principio, usted debe planear y programar todas las actividades del proceso, antes de comenzar a trabajar con ellas.

Las principales etapas del modelo en cascada reflejan directamente las actividades fundamentales del desarrollo:

1. *Análisis y definición de requerimientos* Los servicios, las restricciones y las metas del sistema se establecen mediante consulta a los usuarios del sistema. Luego, se definen con detalle y sirven como una especificación del sistema.
2. *Diseño del sistema y del software* El proceso de diseño de sistemas asigna los requerimientos, para sistemas de hardware o de software, al establecer una arquitectura de sistema global. El diseño del software implica identificar y describir las abstracciones fundamentales del sistema de software y sus relaciones.
3. *Implementación y prueba de unidad* Durante esta etapa, el diseño de software se realiza como un conjunto de programas o unidades del programa. La prueba de unidad consiste en verificar que cada unidad cumpla con su especificación.
4. *Integración y prueba de sistema* Las unidades del programa o los programas individuales se integran y prueban como un sistema completo para asegurarse de que se cumplan los requerimientos de software. Después de probarlo, se libera el sistema de software al cliente.
5. *Operación y mantenimiento* Por lo general (aunque no necesariamente), ésta es la fase más larga del ciclo de vida, donde el sistema se instala y se pone en práctica. El mantenimiento incluye corregir los errores que no se detectaron en etapas anteriores del ciclo de vida, mejorar la implementación de las unidades del sistema e incrementar los servicios del sistema conforme se descubren nuevos requerimientos.

En principio, el resultado de cada fase consiste en uno o más documentos que se autorizaron (“firmaron”). La siguiente fase no debe comenzar sino hasta que termine la fase previa. En la práctica, dichas etapas se traslapan y se nutren mutuamente de información. Durante el diseño se identifican los problemas con los requerimientos. En la codificación se descubren problemas de diseño, y así sucesivamente. El proceso de software no es un simple modelo lineal, sino que implica retroalimentación de una fase a otra. Entonces, es posible que los documentos generados en cada fase deban modificarse para reflejar los cambios que se realizan.

Debido a los costos de producción y aprobación de documentos, las iteraciones suelen ser onerosas e implicar un rediseño significativo. Por lo tanto, después de un pequeño número de iteraciones, es normal detener partes del desarrollo, como la especificación, y continuar con etapas de desarrollo posteriores. Los problemas se dejan para una resolución posterior, se ignoran o se programan. Este freno prematuro de los requerimientos quizá signifique que el sistema no hará lo que el usuario desea. También podría conducir a sistemas mal estructurados conforme los problemas de diseño se evadan con la implementación de trucos.

Durante la fase final del ciclo de vida (operación y mantenimiento), el software se pone en servicio. Se descubren los errores y las omisiones en los requerimientos originales del software. Surgen los errores de programa y diseño, y se detecta la necesidad de nueva funcionalidad. Por lo tanto, el sistema debe evolucionar para mantenerse útil. Hacer tales cambios (mantenimiento de software) puede implicar la repetición de etapas anteriores del proceso.



Ingeniería de software de cuarto limpio

Un ejemplo del proceso de desarrollo formal, diseñado originalmente por IBM, es el proceso de cuarto limpio (*cleanroom*). En el proceso de cuarto limpio, cada incremento de software se especifica formalmente y tal especificación se transforma en una implementación. La exactitud del software se demuestra mediante un enfoque formal. No hay prueba de unidad para defectos en el proceso y la prueba del sistema se enfoca en la valoración de la fiabilidad del sistema.

El objetivo del proceso de cuarto limpio es obtener un software con cero defectos, de modo que los sistemas que se entreguen cuenten con un alto nivel de fiabilidad.

<http://www.SoftwareEngineering-9.com/Web/Cleanroom/>

El modelo en cascada es consecuente con otros modelos del proceso de ingeniería y en cada fase se produce documentación. Esto hace que el proceso sea visible, de modo que los administradores monitoricen el progreso contra el plan de desarrollo. Su principal problema es la partición inflexible del proyecto en distintas etapas. Tienen que establecerse compromisos en una etapa temprana del proceso, lo que dificulta responder a los requerimientos cambiantes del cliente.

En principio, el modelo en cascada sólo debe usarse cuando los requerimientos se entiendan bien y sea improbable el cambio radical durante el desarrollo del sistema. Sin embargo, el modelo en cascada refleja el tipo de proceso utilizado en otros proyectos de ingeniería. Como es más sencillo emplear un modelo de gestión común durante todo el proyecto, aún son de uso común los procesos de software basados en el modelo en cascada.

Una variación importante del modelo en cascada es el desarrollo de sistemas formales, donde se crea un modelo matemático para una especificación del sistema. Después se corrige este modelo, mediante transformaciones matemáticas que preservan su consistencia en un código ejecutable. Con base en la suposición de que son correctas sus transformaciones matemáticas, se puede aseverar, por lo tanto, que un programa generado de esta forma es consecuente con su especificación.

Los procesos formales de desarrollo, como el que se basa en el método B (Schneider, 2001; Wordsworth, 1996) son muy adecuados para el desarrollo de sistemas que cuenten con rigurosos requerimientos de seguridad, fiabilidad o protección. El enfoque formal simplifica la producción de un caso de protección o seguridad. Esto demuestra a los clientes o reguladores que el sistema en realidad cumple sus requerimientos de protección o seguridad.

Los procesos basados en transformaciones formales se usan por lo general sólo en el desarrollo de sistemas críticos para protección o seguridad. Requieren experiencia especializada. Para la mayoría de los sistemas, este proceso no ofrece costo/beneficio significativos sobre otros enfoques en el desarrollo de sistemas.

2.1.2 Desarrollo incremental

El desarrollo incremental se basa en la idea de diseñar una implementación inicial, exponer ésta al comentario del usuario, y luego desarrollarla en sus diversas versiones hasta producir un sistema adecuado (figura 2.2). Las actividades de especificación, desarrollo

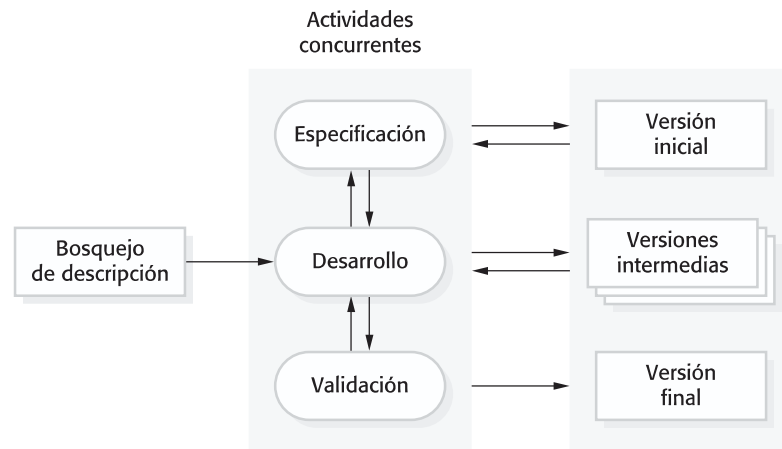


Figura 2.2 Desarrollo incremental

y validación están entrelazadas en vez de separadas, con rápida retroalimentación a través de las actividades.

El desarrollo de software incremental, que es una parte fundamental de los enfoques ágiles, es mejor que un enfoque en cascada para la mayoría de los sistemas empresariales, de comercio electrónico y personales. El desarrollo incremental refleja la forma en que se resuelven problemas. Rara vez se trabaja por adelantado una solución completa del problema, más bien se avanza en una serie de pasos hacia una solución y se retrocede cuando se detecta que se cometieron errores. Al desarrollar el software de manera incremental, resulta más barato y fácil realizar cambios en el software conforme éste se diseña.

Cada incremento o versión del sistema incorpora algunas de las funciones que necesita el cliente. Por lo general, los primeros incrementos del sistema incluyen la función más importante o la más urgente. Esto significa que el cliente puede evaluar el desarrollo del sistema en una etapa relativamente temprana, para constatar si se entrega lo que se requiere. En caso contrario, sólo el incremento actual debe cambiarse y, posiblemente, definir una nueva función para incrementos posteriores.

Comparado con el modelo en cascada, el desarrollo incremental tiene tres beneficios importantes:

1. Se reduce el costo de adaptar los requerimientos cambiantes del cliente. La cantidad de análisis y la documentación que tiene que reelaborarse son mucho menores de lo requerido con el modelo en cascada.
2. Es más sencillo obtener retroalimentación del cliente sobre el trabajo de desarrollo que se realizó. Los clientes pueden comentar las demostraciones del software y darse cuenta de cuánto se ha implementado. Los clientes encuentran difícil juzgar el avance a partir de documentos de diseño de software.
3. Es posible que sea más rápida la entrega e implementación de software útil al cliente, aun si no se ha incluido toda la funcionalidad. Los clientes tienen posibilidad de usar y ganar valor del software más temprano de lo que sería posible con un proceso en cascada.



Problemas con el desarrollo incremental

Aunque el desarrollo incremental tiene muchas ventajas, no está exento de problemas. La principal causa de la dificultad es el hecho de que las grandes organizaciones tienen procedimientos burocráticos que han evolucionado con el tiempo y pueden suscitar falta de coordinación entre dichos procedimientos y un proceso iterativo o ágil más informal.

En ocasiones, tales procedimientos se hallan ahí por buenas razones: por ejemplo, pueden existir procedimientos para garantizar que el software implementa de manera adecuada regulaciones externas (en Estados Unidos, por ejemplo, las regulaciones de contabilidad Sarbanes-Oxley). El cambio de tales procedimientos podría resultar imposible, de manera que los conflictos son inevitables.

<http://www.SoftwareEngineering-9.com/Web/IncrementalDev/>

El desarrollo incremental ahora es en cierta forma el enfoque más común para el desarrollo de sistemas de aplicación. Este enfoque puede estar basado en un plan, ser ágil o, más usualmente, una mezcla de dichos enfoques. En un enfoque basado en un plan se identifican por adelantado los incrementos del sistema; si se adopta un enfoque ágil, se detectan los primeros incrementos, aunque el desarrollo de incrementos posteriores depende del avance y las prioridades del cliente.

Desde una perspectiva administrativa, el enfoque incremental tiene dos problemas:

1. El proceso no es visible. Los administradores necesitan entregas regulares para medir el avance. Si los sistemas se desarrollan rápidamente, resulta poco efectivo en términos de costos producir documentos que reflejen cada versión del sistema.
2. La estructura del sistema tiende a degradarse conforme se tienen nuevos incrementos. A menos que se gaste tiempo y dinero en la refactorización para mejorar el software, el cambio regular tiende a corromper su estructura. La incorporación de más cambios de software se vuelve cada vez más difícil y costosa.

Los problemas del desarrollo incremental se tornan particularmente agudos para sistemas grandes, complejos y de larga duración, donde diversos equipos desarrollan diferentes partes del sistema. Los grandes sistemas necesitan de un marco o una arquitectura estable y es necesario definir con claridad, respecto a dicha arquitectura, las responsabilidades de los distintos equipos que trabajan en partes del sistema. Esto debe planearse por adelantado en vez de desarrollarse de manera incremental.

Se puede desarrollar un sistema incremental y exponerlo a los clientes para su comentario, sin realmente entregarlo e implementarlo en el entorno del cliente. La entrega y la implementación incrementales significan que el software se usa en procesos operacionales reales. Esto no siempre es posible, ya que la experimentación con un nuevo software llega a alterar los procesos empresariales normales. En la sección 2.3.2 se estudian las ventajas y desventajas de la entrega incremental.

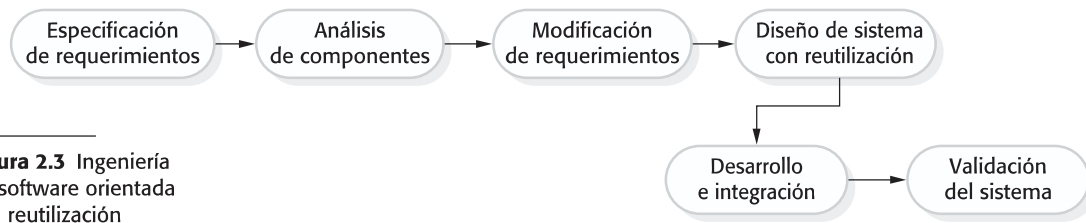


Figura 2.3 Ingeniería de software orientada a la reutilización

2.1.3 Ingeniería de software orientada a la reutilización

En la mayoría de los proyectos de software hay cierta reutilización de software. Sucede con frecuencia de manera informal, cuando las personas que trabajan en el proyecto conocen diseños o códigos que son similares a lo que se requiere. Los buscan, los modifican según se necesite y los incorporan en sus sistemas.

Esta reutilización informal ocurre independientemente del proceso de desarrollo que se emplee. Sin embargo, en el siglo XXI, los procesos de desarrollo de software que se enfocaban en la reutilización de software existente se utilizan ampliamente. Los enfoques orientados a la reutilización se apoyan en una gran base de componentes de software reusable y en la integración de marcos para la composición de dichos componentes. En ocasiones, tales componentes son sistemas por derecho propio (sistemas comerciales, off-the-shelf o COTS) que pueden mejorar la funcionalidad específica, como el procesador de textos o la hoja de cálculo.

En la figura 2.3 se muestra un modelo del proceso general para desarrollo basado en reutilización. Aunque la etapa inicial de especificación de requerimientos y la etapa de validación se comparan con otros procesos de software en un proceso orientado a la reutilización, las etapas intermedias son diferentes. Dichas etapas son:

1. **Análisis de componentes** Dada la especificación de requerimientos, se realiza una búsqueda de componentes para implementar dicha especificación. Por lo general, no hay coincidencia exacta y los componentes que se usan proporcionan sólo parte de la funcionalidad requerida.
2. **Modificación de requerimientos** Durante esta etapa se analizan los requerimientos usando información de los componentes descubiertos. Luego se modifican para reflejar los componentes disponibles. Donde las modificaciones son imposibles, puede regresarse a la actividad de análisis de componentes para buscar soluciones alternativas.
3. **Diseño de sistema con reutilización** Durante esta fase se diseña el marco conceptual del sistema o se reutiliza un marco conceptual existente. Los creadores toman en cuenta los componentes que se reutilizan y organizan el marco de referencia para atenderlo. Es posible que deba diseñarse algo de software nuevo, si no están disponibles los componentes reutilizables.
4. **Desarrollo e integración** Se diseña el software que no puede procurarse de manera externa, y se integran los componentes y los sistemas COTS para crear el nuevo sistema. La integración del sistema, en este modelo, puede ser parte del proceso de desarrollo, en vez de una actividad independiente.

Existen tres tipos de componentes de software que pueden usarse en un proceso orientado a la reutilización:

1. Servicios Web que se desarrollan en concordancia para atender servicios estándares y que están disponibles para la invocación remota.
2. Colecciones de objetos que se desarrollan como un paquete para su integración con un marco de componentes como .NET o J2EE.
3. Sistemas de software independientes que se configuran para usar en un entorno particular.

La ingeniería de software orientada a la reutilización tiene la clara ventaja de reducir la cantidad de software a desarrollar y, por lo tanto, la de disminuir costos y riesgos; por lo general, también conduce a entregas más rápidas del software. Sin embargo, son inevitables los compromisos de requerimientos y esto conduciría hacia un sistema que no cubra las necesidades reales de los usuarios. Más aún, se pierde algo de control sobre la evolución del sistema, conforme las nuevas versiones de los componentes reutilizables no estén bajo el control de la organización que los usa.

La reutilización de software es muy importante y en la tercera parte del libro se dedican varios capítulos a este tema. En el capítulo 16 se tratan los conflictos generales de la reutilización de software y la reutilización de COTS, en los capítulos 17 y 18 se estudia la ingeniería de software basada en componentes, y en el capítulo 19 se explican los sistemas orientados al servicio.

2.2 Actividades del proceso

Los procesos de software real son secuencias entrelazadas de actividades técnicas, colaborativas y administrativas con la meta general de especificar, diseñar, implementar y probar un sistema de software. Los desarrolladores de software usan en su trabajo diferentes herramientas de software. Las herramientas son útiles particularmente para dar apoyo a la edición de distintos tipos de documento y para manejar el inmenso volumen de información detallada que se reproduce en un gran proyecto de software.

Las cuatro actividades básicas de proceso de especificación, desarrollo, validación y evolución se organizan de diversa manera en diferentes procesos de desarrollo. En el modelo en cascada se organizan en secuencia, mientras que se entrelazan en el desarrollo incremental. La forma en que se llevan a cabo estas actividades depende del tipo de software, del personal y de la inclusión de estructuras organizativas. En la programación extrema, por ejemplo, las especificaciones se escriben en tarjetas. Las pruebas son ejecutables y se desarrollan antes del programa en sí. La evolución incluye la reestructuración o refactorización sustancial del sistema.

2.2.1 Especificación del software

La especificación del software o la ingeniería de requerimientos consisten en el proceso de comprender y definir qué servicios se requieren del sistema, así como la identificación de las restricciones sobre la operación y el desarrollo del sistema. La ingeniería de requerimientos es una etapa particularmente crítica del proceso de software, ya que los



Herramientas de desarrollo de software

Las herramientas de desarrollo del software (llamadas en ocasiones herramientas de Ingeniería de Software Asistido por Computadora o CASE, por las siglas de *Computer-Aided Software Engineering*) son programas usados para apoyar las actividades del proceso de la ingeniería de software. En consecuencia, estas herramientas incluyen editores de diseño, diccionarios de datos, compiladores, depuradores (*debuggers*), herramientas de construcción de sistema, etcétera.

Las herramientas de software ofrecen apoyo de proceso al automatizar algunas actividades del proceso y brindar información sobre el software que se desarrolla. Los ejemplos de actividades susceptibles de automatizarse son:

- Desarrollo de modelos de sistemas gráficos, como parte de la especificación de requerimientos o del diseño del software.
- Generación de código a partir de dichos modelos de sistemas gráficos.
- Producción de interfaces de usuario a partir de una descripción de interfaz gráfica, creada por el usuario de manera interactiva.
- Depuración del programa mediante el suministro de información sobre un programa que se ejecuta.
- Traducción automatizada de programas escritos, usando una versión anterior de un lenguaje de programación para tener una versión más reciente.

Las herramientas pueden combinarse en un marco llamado ambiente de desarrollo interactivo o IDE (por las siglas de *Interactive Development Environment*). Esto ofrece un conjunto común de facilidades, que usan las herramientas para comunicarse y operar con mayor destreza en una forma integrada. El ECLIPSE IDE se usa ampliamente y se diseñó para incorporar muchos tipos diferentes de herramientas de software.

<http://www.SoftwareEngineering-9.com/Web/CASE/>

errores en esta etapa conducen de manera inevitable a problemas posteriores tanto en el diseño como en la implementación del sistema.

El proceso de ingeniería de requerimientos (figura 2.4) se enfoca en producir un documento de requerimientos convenido que especifique los requerimientos de los interesados que cumplirá el sistema. Por lo general, los requerimientos se presentan en dos niveles de detalle. Los usuarios finales y clientes necesitan un informe de requerimientos de alto nivel; los desarrolladores de sistemas precisan una descripción más detallada del sistema.

Existen **cuatro actividades principales en el proceso de ingeniería de requerimientos:**

1. **Estudio de factibilidad** Se realiza una estimación sobre si las necesidades identificadas del usuario se cubren con las actuales tecnologías de software y hardware. El estudio considera si el sistema propuesto tendrá un costo-beneficio desde un punto de vista empresarial, y si éste puede desarrollarse dentro de las restricciones presupuestales existentes. Un estudio de factibilidad debe ser rápido y relativamente barato. El resultado debe informar la decisión respecto a si se continúa o no continúa con un análisis más detallado.
2. **Obtención y análisis de requerimientos** Éste es el proceso de derivar los requerimientos del sistema mediante observación de los sistemas existentes, discusiones con los usuarios y proveedores potenciales, análisis de tareas, etcétera. Esto puede incluir el desarrollo de uno o más modelos de sistemas y prototipos, lo que ayuda a entender el sistema que se va a especificar.

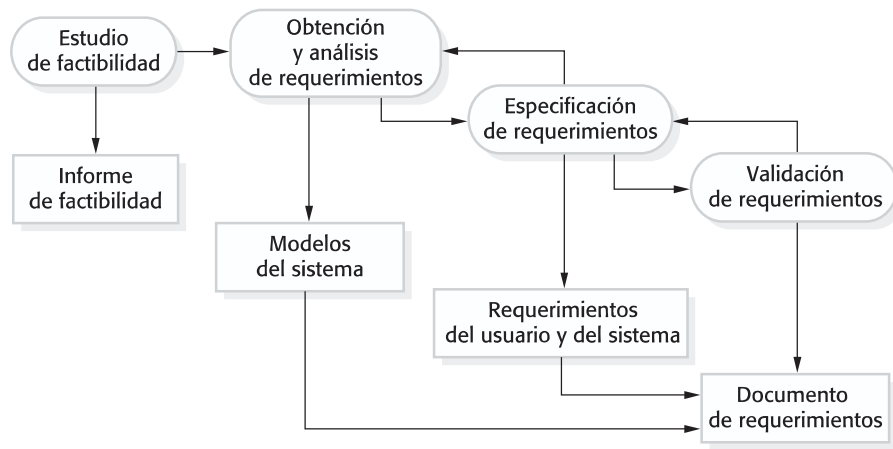


Figura 2.4 Proceso de ingeniería de requerimientos

3. **Especificación de requerimientos** Consiste en la actividad de transcribir la información recopilada durante la actividad de análisis, en un documento que define un conjunto de requerimientos. En este documento se incluyen dos clases de requerimientos. Los requerimientos del usuario son informes abstractos de requerimientos del sistema para el cliente y el usuario final del sistema; y los requerimientos de sistema son una descripción detallada de la funcionalidad a ofrecer.
4. **Validación de requerimientos** Esta actividad verifica que los requerimientos sean realistas, coherentes y completos. Durante este proceso es inevitable descubrir errores en el documento de requerimientos. En consecuencia, deberían modificarse con la finalidad de corregir dichos problemas.

Desde luego, las actividades en el proceso de requerimientos no se realizan simplemente en una secuencia estricta. El análisis de requerimientos continúa durante la definición y especificación, y a lo largo del proceso salen a la luz nuevos requerimientos; por lo tanto, las actividades de análisis, definición y especificación están vinculadas. En los métodos ágiles, como programación extrema, los requerimientos se desarrollan de manera incremental según las prioridades del usuario, en tanto que la obtención de requerimientos proviene de los usuarios que son parte del equipo de desarrollo.

2.2.2 Diseño e implementación del software

La etapa de implementación de desarrollo del software corresponde al proceso de convertir una especificación del sistema en un sistema ejecutable. Siempre incluye procesos de diseño y programación de software, aunque también puede involucrar la corrección en la especificación del software, si se utiliza un enfoque incremental de desarrollo.

Un diseño de software se entiende como una descripción de la estructura del software que se va a implementar, los modelos y las estructuras de datos utilizados por el sistema, las interfaces entre componentes del sistema y, en ocasiones, los algoritmos usados. Los diseñadores no llegan inmediatamente a una creación terminada, sino que desarrollan el diseño de manera iterativa. Agregan formalidad y detalle conforme realizan su diseño con *backtracking* (vuelta atrás) constante para corregir diseños anteriores.

La figura 2.5 es un modelo abstracto de este proceso, que ilustra las entradas al proceso de diseño, las actividades del proceso y los documentos generados como salidas de este proceso.

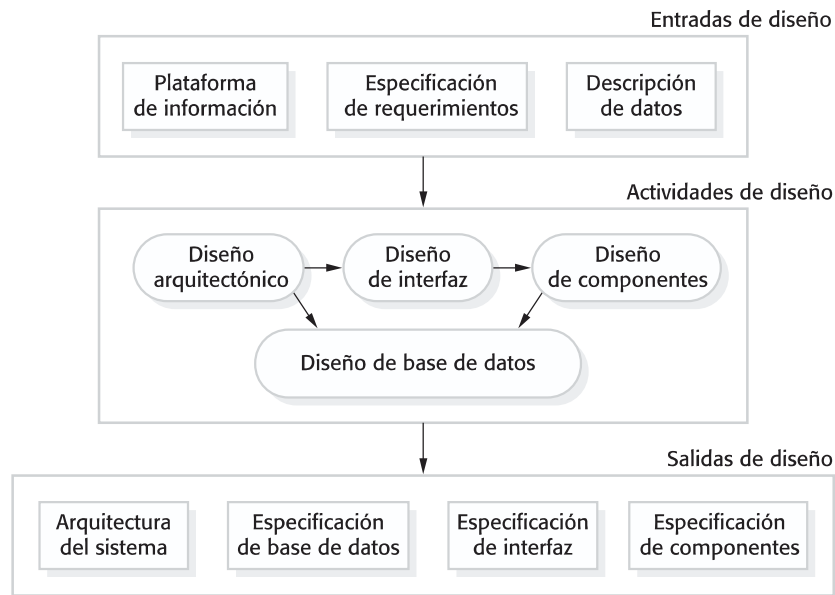


Figura 2.5 Modelo general del proceso de diseño

El diagrama sugiere que las etapas del proceso de diseño son secuenciales. De hecho, las actividades de proceso de diseño están vinculadas. En todos los procesos de diseño es inevitable la retroalimentación de una etapa a otra y la consecuente reelaboración del diseño.

La mayoría del software tiene interfaz junto con otros sistemas de software. En ellos se incluyen sistema operativo, base de datos, *middleware* y otros sistemas de aplicación. Éstos constituyen la “plataforma de software”, es decir, el entorno donde se ejecutará el software. La información sobre esta plataforma es una entrada esencial al proceso de diseño, así que los diseñadores tienen que decidir la mejor forma de integrarla con el entorno de software. La especificación de requerimientos es una descripción de la funcionalidad que debe brindar el software, en conjunción con sus requerimientos de rendimiento y confiabilidad. Si el sistema debe procesar datos existentes, entonces en la especificación de la plataforma se incluirá la descripción de tales datos; de otro modo, la descripción de los datos será una entrada al proceso de diseño, de manera que se defina la organización del sistema de datos.

Las actividades en el proceso de diseño varían dependiendo del tipo de sistema a desarrollar. Por ejemplo, los sistemas de tiempo real precisan del diseño de temporización, pero sin incluir una base de datos, por lo que no hay que integrar un diseño de base de datos. La figura 2.5 muestra cuatro actividades que podrían formar parte del proceso de diseño para sistemas de información:

1. *Diseño arquitectónico*, aquí se identifica la estructura global del sistema, los principales componentes (llamados en ocasiones subsistemas o módulos), sus relaciones y cómo se distribuyen.
2. *Diseño de interfaz*, en éste se definen las interfaces entre los componentes de sistemas. Esta especificación de interfaz no tiene que presentar ambigüedades. Con una interfaz precisa, es factible usar un componente sin que otros tengan que saber cómo se implementó. Una vez que se acuerdan las especificaciones de interfaz, los componentes se diseñan y se desarrollan de manera concurrente.



Métodos estructurados

Los métodos estructurados son un enfoque al diseño de software donde se definen los modelos gráficos que hay que desarrollar, como parte del proceso de diseño. El método también define un proceso para diseñar los modelos y las reglas que se aplican a cada tipo de modelo. Los métodos estructurados conducen a documentación estandarizada para un sistema y son muy útiles al ofrecer un marco de desarrollo para los creadores de software con menor experiencia.

<http://www.SoftwareEngineering-9.com/Web/Structured-methods/>

3. *Diseño de componentes*, en él se toma cada componente del sistema y se diseña cómo funcionará. Esto puede ser un simple dato de la funcionalidad que se espera implementar, y al programador se le deja el diseño específico. Como alternativa, habría una lista de cambios a realizar sobre un componente que se reutiliza o sobre un modelo de diseño detallado. El modelo de diseño sirve para generar en automático una implementación.
4. *Diseño de base de datos*, donde se diseñan las estructuras del sistema de datos y cómo se representarán en una base de datos. De nuevo, el trabajo aquí depende de si una base de datos se reutilizará o se creará una nueva.

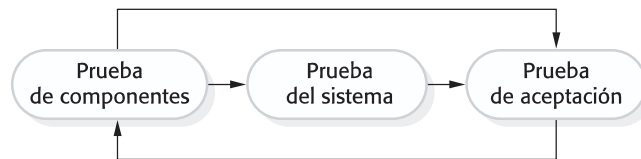
Tales actividades conducen a un conjunto de salidas de diseño, que también se muestran en la figura 2.5. El detalle y la representación de las mismas varían considerablemente. Para sistemas críticos, deben producirse documentos de diseño detallados que establezcan descripciones exactas del sistema. Si se usa un enfoque dirigido por un modelo, dichas salidas serían sobre todo diagramas. Donde se usen métodos ágiles de desarrollo, las salidas del proceso de diseño no podrían ser documentos de especificación separados, sino que tendrían que representarse en el código del programa.

Los métodos estructurados para el diseño se desarrollaron en las décadas de 1970 y 1980, y fueron precursores del UML y del diseño orientado a objetos (Budgen, 2003). Se apoyan en la producción de modelos gráficos del sistema y, en muchos casos, generan instantáneamente un código a partir de dichos modelos. El desarrollo dirigido por modelo (MDD) o la ingeniería dirigida por modelo (Schmidt, 2006), donde se crean modelos de software a diferentes niveles de abstracción, es una evolución de los métodos estructurados. En el MDD hay mayor énfasis en los modelos arquitectónicos con una separación entre modelos abstractos independientes de implementación y modelos específicos de implementación. Los modelos se desarrollan con detalle suficiente, de manera que el sistema ejecutable puede generarse a partir de ellos. En el capítulo 5 se estudia este enfoque de desarrollo.

El diseño de un programa para implementar el sistema se sigue naturalmente de los procesos de elaboración del sistema. Aunque algunas clases de programa, como los sistemas críticos para la seguridad, por lo general se diseñan con detalle antes de comenzar cualquier implementación, es más común que se entrelacen en etapas posteriores del diseño y el desarrollo del programa. Las herramientas de desarrollo de software se usan para generar un programa de “esqueleto” a partir de un diseño. Esto incluye un código para definir e implementar interfaces y, en muchos casos, el desarrollador sólo necesita agregar detalles de la operación de cada componente del programa.

La programación es una actividad personal y no hay un proceso que se siga de manera general. Algunos programadores comienzan con componentes que entienden, los desarrollan y, luego, cambian hacia componentes que entienden menos. Otros toman el enfoque opuesto,

Figura 2.6 Etapas de pruebas



y dejan hasta el último los componentes familiares, porque saben cómo diseñarlos. A algunos desarrolladores les agrada definir con anticipación datos en el proceso, que luego usan para impulsar el desarrollo del programa; otros dejan datos sin especificar tanto como sea posible.

Por lo general, los programadores realizan algunas pruebas del código que desarrollaron. Esto revela con frecuencia defectos del programa que deben eliminarse del programa. A esta actividad se le llama depuración (*debugging*). La prueba de defectos y la depuración son procesos diferentes. La primera establece la existencia de defectos, en tanto que la segunda se dedica a localizar y corregir dichos defectos.

Cuando se depura, uno debe elaborar una hipótesis sobre el comportamiento observable del programa y, luego, poner a prueba dichas hipótesis con la esperanza de encontrar la falla que causó la salida anómala. Poner a prueba las hipótesis quizá requiera rastrear manualmente el código del programa; o bien, tal vez se necesiten nuevos casos de prueba para localizar el problema. Con la finalidad de apoyar el proceso de depuración, se deben utilizar herramientas interactivas que muestren valores intermedios de las variables del programa, así como el rastro de las instrucciones ejecutadas.

2.2.3 Validación de software

La validación de software o, más generalmente, su verificación y validación (V&V), se crea para mostrar que un sistema cumple tanto con sus especificaciones como con las expectativas del cliente. Las pruebas del programa, donde el sistema se ejecuta a través de datos de prueba simulados, son la principal técnica de validación. Esta última también puede incluir procesos de comprobación, como inspecciones y revisiones en cada etapa del proceso de software, desde la definición de requerimientos del usuario hasta el desarrollo del programa. Dada la predominancia de las pruebas, se incurre en la mayoría de los costos de validación durante la implementación y después de ésta.

Con excepción de los programas pequeños, los sistemas no deben ponerse a prueba como una unidad monolítica. La figura 2.6 muestra un proceso de prueba en tres etapas, donde los componentes del sistema se ponen a prueba; luego, se hace lo mismo con el sistema integrado y, finalmente, el sistema se pone a prueba con los datos del cliente. De manera ideal, los defectos de los componentes se detectan oportunamente en el proceso, en tanto que los problemas de interfaz se localizan cuando el sistema se integra. Sin embargo, conforme se descubran los defectos, el programa deberá depurarse y esto quizá requiera la repetición de otras etapas en el proceso de pruebas. Los errores en los componentes del programa pueden salir a la luz durante las pruebas del sistema. En consecuencia, el proceso es iterativo, con información retroalimentada desde etapas posteriores hasta las partes iniciales del proceso.

Las etapas en el proceso de pruebas son:

1. *Prueba de desarrollo* Las personas que desarrollan el sistema ponen a prueba los componentes que constituyen el sistema. Cada componente se prueba de manera independiente, es decir, sin otros componentes del sistema. Éstos pueden ser simples

entidades, como funciones o clases de objeto, o agrupamientos coherentes de dichas entidades. Por lo general, se usan herramientas de automatización de pruebas, como JUnit (Massol y Husted, 2003), que pueden volver a correr pruebas de componentes cuando se crean nuevas versiones del componente.

2. *Pruebas del sistema* Los componentes del sistema se integran para crear un sistema completo. Este proceso tiene la finalidad de descubrir errores que resulten de interacciones no anticipadas entre componentes y problemas de interfaz de componente, así como de mostrar que el sistema cubre sus requerimientos funcionales y no funcionales, y poner a prueba las propiedades emergentes del sistema. Para sistemas grandes, esto puede ser un proceso de múltiples etapas, donde los componentes se conjuntan para formar subsistemas que se ponen a prueba de manera individual, antes de que dichos subsistemas se integren para establecer el sistema final.
3. *Pruebas de aceptación* Ésta es la etapa final en el proceso de pruebas, antes de que el sistema se acepte para uso operacional. El sistema se pone a prueba con datos suministrados por el cliente del sistema, en vez de datos de prueba simulados. Las pruebas de aceptación revelan los errores y las omisiones en la definición de requerimientos del sistema, ya que los datos reales ejercitan el sistema en diferentes formas a partir de los datos de prueba. Asimismo, las pruebas de aceptación revelan problemas de requerimientos, donde las instalaciones del sistema en realidad no cumplan las necesidades del usuario o cuando sea inaceptable el rendimiento del sistema.

Por lo general, los procesos de desarrollo y de pruebas de componentes están entrelazados. Los programadores construyen sus propios datos de prueba y experimentan el código de manera incremental conforme lo desarrollan. Éste es un enfoque económicamente sensible, ya que el programador conoce el componente y, por lo tanto, es el más indicado para generar casos de prueba.

Si se usa un enfoque incremental para el desarrollo, cada incremento debe ponerse a prueba conforme se diseña, y tales pruebas se basan en los requerimientos para dicho incremento. En programación extrema, las pruebas se desarrollan junto con los requerimientos antes de comenzar el desarrollo. Esto ayuda a los examinadores y desarrolladores a comprender los requerimientos, y garantiza que no haya demoras conforme se creen casos de prueba.

Cuando se usa un proceso de software dirigido por un plan (como en el desarrollo de sistemas críticos), las pruebas se realizan mediante un conjunto de planes de prueba. Un equipo independiente de examinadores trabaja con base en dichos planes de prueba preformulados, que se desarrollaron a partir de la especificación y el diseño del sistema. La figura 2.7 ilustra cómo se vinculan los planes de prueba entre las actividades de pruebas y desarrollo. A esto se le conoce en ocasiones como modelo V de desarrollo (colóquelo de lado para distinguir la V).

En ocasiones, a las pruebas de aceptación se les identifica como “pruebas alfa”. Los sistemas a la medida se desarrollan sólo para un cliente. El proceso de prueba alfa continúa hasta que el desarrollador del sistema y el cliente estén de acuerdo en que el sistema entregado es una implementación aceptable de los requerimientos.

Cuando un sistema se marca como producto de software, se utiliza con frecuencia un proceso de prueba llamado “prueba beta”. Ésta incluye entregar un sistema a algunos clientes potenciales que están de acuerdo con usar ese sistema. Ellos reportan los problemas a los desarrolladores del sistema. Dicho informe expone el producto a uso real y detecta errores que no anticiparon los constructores del sistema. Después de esta retroalimentación, el sistema se modifica y libera, ya sea para más pruebas beta o para su venta general.

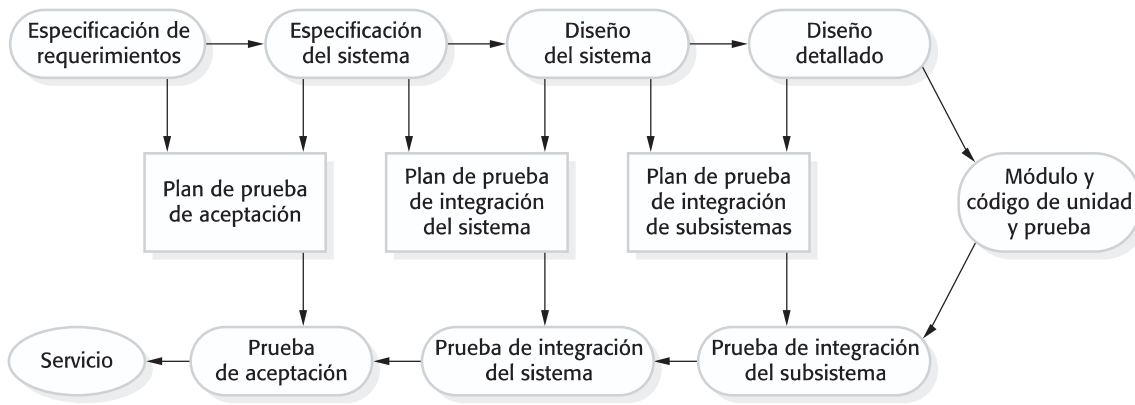


Figura 2.7
Probando
fases en un
proceso de
software dirigido
por un plan

2.2.4 Evolución del software

La flexibilidad de los sistemas de software es una de las razones principales por las que cada vez más software se incorpora en los sistemas grandes y complejos. Una vez tomada la decisión de fabricar hardware, resulta muy costoso hacer cambios a su diseño. Sin embargo, en cualquier momento durante o después del desarrollo del sistema, pueden hacerse cambios al software. Incluso los cambios mayores son todavía más baratos que los correspondientes cambios al hardware del sistema.

En la historia, siempre ha habido división entre el proceso de desarrollo del software y el proceso de evolución del software (mantenimiento de software). Las personas consideran el desarrollo de software como una actividad creativa, en la cual se diseña un sistema de software desde un concepto inicial y a través de un sistema de trabajo. No obstante, consideran en ocasiones el mantenimiento del software como insulso y poco interesante. Aunque en la mayoría de los casos los costos del mantenimiento son varias veces los costos iniciales de desarrollo, los procesos de mantenimiento se consideran en ocasiones como menos desafiantes que el desarrollo de software original.

Esta distinción entre desarrollo y mantenimiento es cada vez más irrelevante. Es muy difícil que cualquier sistema de software sea un sistema completamente nuevo, y tiene mucho más sentido ver el desarrollo y el mantenimiento como un continuo. En lugar de dos procesos separados, es más realista pensar en la ingeniería de software como un proceso evolutivo (figura 2.8), donde el software cambia continuamente a lo largo de su vida, en función de los requerimientos y las necesidades cambiantes del cliente.

2.3 Cómo enfrentar el cambio

El cambio es inevitable en todos los grandes proyectos de software. Los requerimientos del sistema varían conforme la empresa procura que el sistema responda a presiones externas y se modifican las prioridades administrativas. A medida que se ponen a disposición nuevas tecnologías, surgen nuevas posibilidades de diseño e implementación. Por ende, cualquiera que sea el modelo del proceso de software utilizado, es esencial que ajuste los cambios al software a desarrollar.

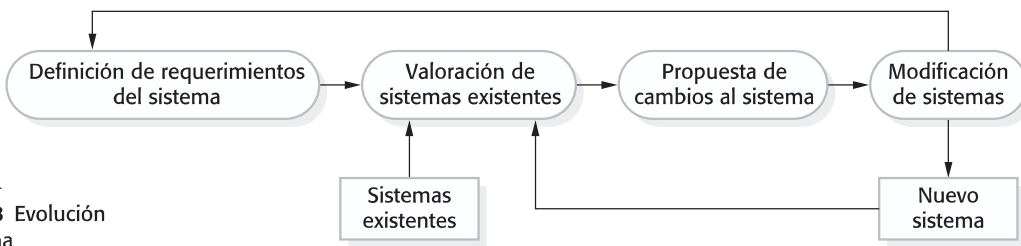


Figura 2.8 Evolución del sistema

El cambio se agrega a los costos del desarrollo de software debido a que, por lo general, significa que el trabajo ya terminado debe volver a realizarse. A esto se le llama rehacer. Por ejemplo, si se analizaron las relaciones entre los requerimientos en un sistema y se identifican nuevos requerimientos, parte o todo el análisis de requerimientos tiene que repetirse. Entonces, es necesario rediseñar el sistema para entregar los nuevos requerimientos, cambiar cualquier programa que se haya desarrollado y volver a probar el sistema.

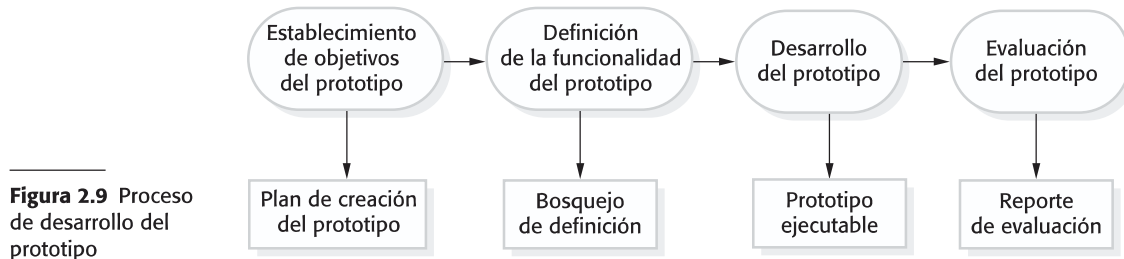
Existen dos enfoques relacionados que se usan para reducir los costos del rehacer:

1. Evitar el cambio, donde el proceso de software incluye actividades que anticipan cambios posibles antes de requerirse la labor significativa de rehacer. Por ejemplo, puede desarrollarse un sistema prototipo para demostrar a los clientes algunas características clave del sistema. Ellos podrán experimentar con el prototipo y refinar sus requerimientos, antes de comprometerse con mayores costos de producción de software.
2. Tolerancia al cambio, donde el proceso se diseña de modo que los cambios se ajusten con un costo relativamente bajo. Por lo general, esto comprende algunas formas de desarrollo incremental. Los cambios propuestos pueden implementarse en incrementos que aún no se desarrollan. Si no es posible, entonces tal vez sólo un incremento (una pequeña parte del sistema) tendría que alterarse para incorporar el cambio.

En esta sección se estudian dos formas de enfrentar el cambio y los requerimientos cambiantes del sistema. Se trata de lo siguiente:

1. Prototipo de sistema, donde rápidamente se desarrolla una versión del sistema o una parte del mismo, para comprobar los requerimientos del cliente y la factibilidad de algunas decisiones de diseño. Esto apoya el hecho de evitar el cambio, al permitir que los usuarios experimenten con el sistema antes de entregarlo y así refinar sus requerimientos. Como resultado, es probable que se reduzca el número de propuestas de cambio de requerimientos posterior a la entrega.
2. Entrega incremental, donde los incrementos del sistema se entregan al cliente para su comentario y experimentación. Esto apoya tanto al hecho de evitar el cambio como a tolerar el cambio. Por un lado, evita el compromiso prematuro con los requerimientos para todo el sistema y, por otro, permite la incorporación de cambios en incrementos mayores a costos relativamente bajos.

La noción de refactorización, esto es, el mejoramiento de la estructura y organización de un programa, es también un mecanismo importante que apoya la tolerancia al cambio. Este tema se explica en el capítulo 3, que se ocupa de los métodos ágiles.



2.3.1 Creación del prototipo

Un prototipo es una versión inicial de un sistema de software que se usa para demostrar conceptos, tratar opciones de diseño y encontrar más sobre el problema y sus posibles soluciones. El rápido desarrollo iterativo del prototipo es esencial, de modo que se controlen los costos, y los interesados en el sistema experimenten por anticipado con el prototipo durante el proceso de software.

Un prototipo de software se usa en un proceso de desarrollo de software para contribuir a anticipar los cambios que se requieran:

1. En el proceso de ingeniería de requerimientos, un prototipo ayuda con la selección y validación de requerimientos del sistema.
2. En el proceso de diseño de sistemas, un prototipo sirve para buscar soluciones específicas de software y apoyar el diseño de interfaces del usuario.

Los prototipos del sistema permiten a los usuarios ver qué tan bien el sistema apoya su trabajo. Pueden obtener nuevas ideas para requerimientos y descubrir áreas de fortalezas y debilidades en el software. Entonces, proponen nuevos requerimientos del sistema. Más aún, conforme se desarrolla el prototipo, quizá se revelen errores y omisiones en los requerimientos propuestos. Una función descrita en una especificación puede parecer útil y bien definida. Sin embargo, cuando dicha función se combina con otras operaciones, los usuarios descubren frecuentemente que su visión inicial era incorrecta o estaba incompleta. Entonces, se modifica la especificación del sistema con la finalidad de reflejar su nueva comprensión de los requerimientos.

Mientras se elabora el sistema para la realización de experimentos de diseño, un prototipo del mismo sirve para comprobar la factibilidad de un diseño propuesto. Por ejemplo, puede crearse un prototipo del diseño de una base de datos y ponerse a prueba, con el objetivo de comprobar que soporta de forma eficiente el acceso de datos para las consultas más comunes del usuario. Asimismo, la creación de prototipos es una parte esencial del proceso de diseño de interfaz del usuario. Debido a la dinámica natural de las interfaces de usuario, las descripciones textuales y los diagramas no son suficientemente buenos para expresar los requerimientos de la interfaz del usuario. Por lo tanto, la creación rápida de prototipos con la participación del usuario final es la única forma sensible para desarrollar interfaces de usuario gráficas para sistemas de software.

En la figura 2.9 se muestra un modelo del proceso para desarrollo de prototipos. Los objetivos de la creación de prototipos deben ser más explícitos desde el inicio del proceso. Esto tendría la finalidad de desarrollar un sistema para un prototipo de la interfaz

del usuario, y diseñar un sistema que valide los requerimientos funcionales del sistema o desarrolle un sistema que demuestre a los administradores la factibilidad de la aplicación. El mismo prototipo no puede cumplir con todos los objetivos, ya que si éstos quedan sin especificar, los administradores o usuarios finales quizá malinterpreten la función del prototipo. En consecuencia, es posible que no obtengan los beneficios esperados del desarrollo del prototipo.

La siguiente etapa del proceso consiste en decidir qué poner y, algo quizá más importante, qué dejar fuera del sistema de prototipo. Para reducir los costos de creación de prototipos y acelerar las fechas de entrega, es posible dejar cierta funcionalidad fuera del prototipo y, también, decidir hacer más flexible los requerimientos no funcionales, como el tiempo de respuesta y la utilización de memoria. El manejo y la gestión de errores pueden ignorarse, a menos que el objetivo del prototipo sea establecer una interfaz de usuario. Además, es posible reducir los estándares de fiabilidad y calidad del programa.

La etapa final del proceso es la evaluación del prototipo. Hay que tomar provisiones durante esta etapa para la capacitación del usuario y usar los objetivos del prototipo para derivar un plan de evaluación. Los usuarios requieren tiempo para sentirse cómodos con un nuevo sistema e integrarse a un patrón normal de uso. Una vez que utilizan el sistema de manera normal, descubren errores y omisiones en los requerimientos.

Un problema general con la creación de prototipos es que quizás el prototipo no se utilice necesariamente en la misma forma que el sistema final. El revisor del prototipo tal vez no sea un usuario típico del sistema. También, podría resultar insuficiente el tiempo de capacitación durante la evaluación del prototipo. Si el prototipo es lento, los evaluadores podrían ajustar su forma de trabajar y evitar aquellas características del sistema con tiempos de respuesta lentos. Cuando se da una mejor respuesta en el sistema final, se puede usar de forma diferente.

En ocasiones, los desarrolladores están presionados por los administradores para entregar prototipos desechables, sobre todo cuando existen demoras en la entrega de la versión final del software. Sin embargo, por lo general esto no es aconsejable:

1. Puede ser imposible corregir el prototipo para cubrir requerimientos no funcionales, como los requerimientos de rendimiento, seguridad, robustez y fiabilidad, ignorados durante el desarrollo del prototipo.
2. El cambio rápido durante el desarrollo significa claramente que el prototipo no está documentado. La única especificación de diseño es el código del prototipo. Esto no es muy bueno para el mantenimiento a largo plazo.
3. Probablemente los cambios realizados durante el desarrollo de prototipos degradarán la estructura del sistema, y este último será difícil y costoso de mantener.
4. Por lo general, durante el desarrollo de prototipos se hacen más flexibles los estándares de calidad de la organización.

Los prototipos no tienen que ser ejecutables para ser útiles. Los modelos en papel de la interfaz de usuario del sistema (Rettig, 1994) pueden ser efectivos para ayudar a los usuarios a refinar un diseño de interfaz y trabajar a través de escenarios de uso. Su desarrollo es muy económico y suelen construirse en pocos días. Una extensión de esta técnica es un prototipo de Mago de Oz, donde sólo se desarrolle la interfaz del usuario. Los usuarios interactúan con esta interfaz, pero sus solicitudes pasan a una persona que los interpreta y les devuelve la respuesta adecuada.

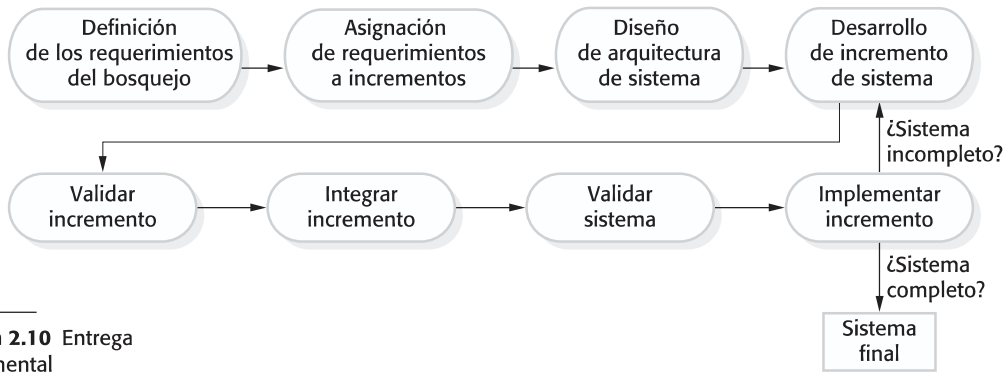


Figura 2.10 Entrega incremental

2.3.2 Entrega incremental

La entrega incremental (figura 2.10) es un enfoque al desarrollo de software donde algunos de los incrementos diseñados se entregan al cliente y se implementan para usarse en un entorno operacional. En un proceso de entrega incremental, los clientes identifican, en un bosquejo, los servicios que proporciona el sistema. Identifican cuáles servicios son más importantes y cuáles son menos significativos para ellos. Entonces, se define un número de incrementos de entrega, y cada incremento proporciona un subconjunto de la funcionalidad del sistema. La asignación de servicios por incrementos depende de la prioridad del servicio, donde los servicios de más alta prioridad se implementan y entregan primero.

Una vez identificados los incrementos del sistema, se definen con detalle los requerimientos de los servicios que se van a entregar en el primer incremento, y se desarrolla ese incremento. Durante el desarrollo, puede haber un mayor análisis de requerimientos para incrementos posteriores, aun cuando se rechacen cambios de requerimientos para el incremento actual.

Una vez completado y entregado el incremento, los clientes lo ponen en servicio. Esto significa que toman la entrega anticipada de la funcionalidad parcial del sistema. Pueden experimentar con el sistema que les ayuda a clarificar sus requerimientos, para posteriores incrementos del sistema. A medida que se completan nuevos incrementos, se integran con los incrementos existentes, de modo que con cada incremento entregado mejore la funcionalidad del sistema.

La entrega incremental tiene algunas ventajas:

1. Los clientes pueden usar los primeros incrementos como prototipos y adquirir experiencia que informe sobre sus requerimientos, para posteriores incrementos del sistema. A diferencia de los prototipos, éstos son parte del sistema real, de manera que no hay reaprendizaje cuando está disponible el sistema completo.
2. Los clientes deben esperar hasta la entrega completa del sistema, antes de ganar valor del mismo. El primer incremento cubre sus requerimientos más críticos, de modo que es posible usar inmediatamente el software.
3. El proceso mantiene los beneficios del desarrollo incremental en cuanto a que debe ser relativamente sencillo incorporar cambios al sistema.
4. Puesto que primero se entregan los servicios de mayor prioridad y luego se integran los incrementos, los servicios de sistema más importantes reciben mayores pruebas.

Esto significa que los clientes tienen menos probabilidad de encontrar fallas de software en las partes más significativas del sistema.

Sin embargo, existen problemas con la entrega incremental:

1. La mayoría de los sistemas requieren de una serie de recursos que se utilizan para diferentes partes del sistema. Dado que los requerimientos no están definidos con detalle sino hasta que se implementa un incremento, resulta difícil identificar recursos comunes que necesiten todos los incrementos.
2. Asimismo, el desarrollo iterativo resulta complicado cuando se diseña un sistema de reemplazo. Los usuarios requieren de toda la funcionalidad del sistema antiguo, ya que es común que no deseen experimentar con un nuevo sistema incompleto. Por lo tanto, es difícil conseguir retroalimentación útil del cliente.
3. La esencia de los procesos iterativos es que la especificación se desarrolla en conjunto con el software. Sin embargo, esto se puede contradecir con el modelo de adquisiciones de muchas organizaciones, donde la especificación completa del sistema es parte del contrato de desarrollo del sistema. En el enfoque incremental, no hay especificación completa del sistema, sino hasta que se define el incremento final. Esto requiere una nueva forma de contrato que los grandes clientes, como las agencias gubernamentales, encontrarían difícil de adoptar.

Existen algunos tipos de sistema donde el desarrollo incremental y la entrega no son el mejor enfoque. Hay sistemas muy grandes donde el desarrollo incluye equipos que trabajan en diferentes ubicaciones, algunos sistemas embebidos donde el software depende del desarrollo de hardware y algunos sistemas críticos donde todos los requerimientos tienen que analizarse para comprobar las interacciones que comprometan la seguridad o protección del sistema.

Estos sistemas, desde luego, enfrentan los mismos problemas de incertidumbre y requerimientos cambiantes. En consecuencia, para solucionar tales problemas y obtener algunos de los beneficios del desarrollo incremental, se utiliza un proceso donde un prototipo del sistema se elabore iterativamente y se utilice como plataforma, para experimentar con los requerimientos y el diseño del sistema. Con la experiencia obtenida del prototipo, pueden concertarse los requerimientos definitivos.

2.3.3 Modelo en espiral de Boehm

Boehm (1988) propuso un marco del proceso de software dirigido por el riesgo (el modelo en espiral), que se muestra en la figura 2.11. Aquí, el proceso de software se representa como una espiral, y no como una secuencia de actividades con cierto retroceso de una actividad a otra. Cada ciclo en la espiral representa una fase del proceso de software. Por ende, el ciclo más interno puede relacionarse con la factibilidad del sistema, el siguiente ciclo con la definición de requerimientos, el ciclo que sigue con el diseño del sistema, etcétera. El modelo en espiral combina el evitar el cambio con la tolerancia al cambio. Lo anterior supone que los cambios son resultado de riesgos del proyecto e incluye actividades de gestión de riesgos explícitas para reducir tales riesgos.

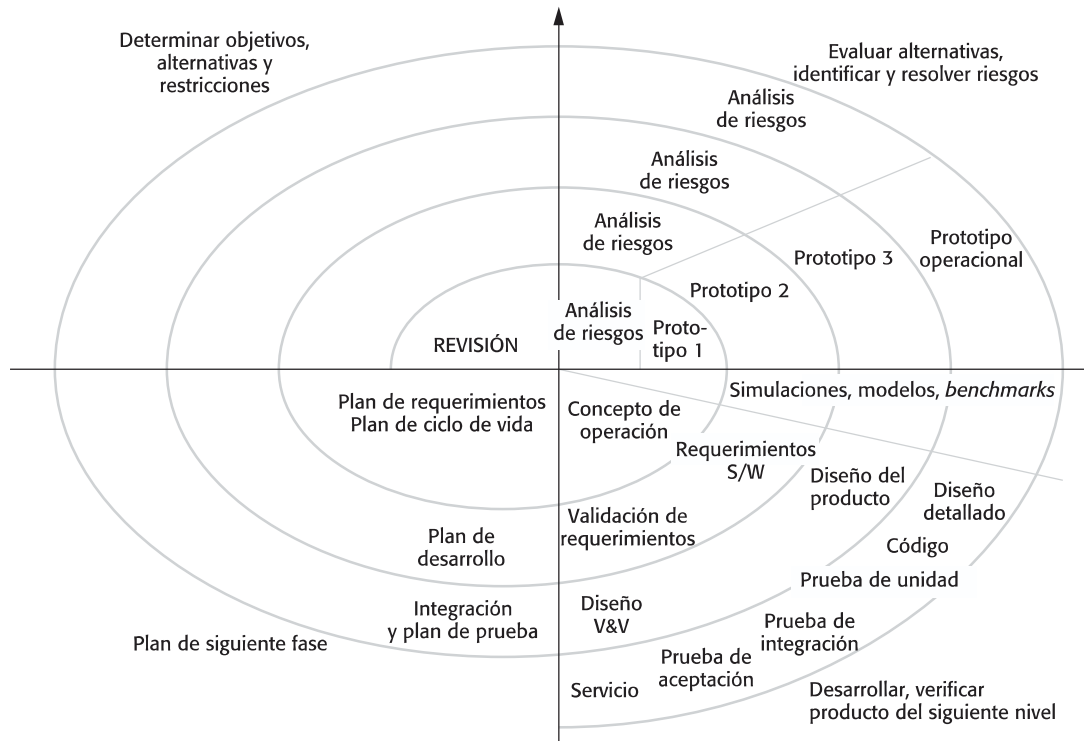


Figura 2.11 Modelo en espiral de Boehm del proceso de software
(© IEEE, 1988)

Cada ciclo en la espiral se divide en cuatro sectores:

1. *Establecimiento de objetivos* Se definen objetivos específicos para dicha fase del proyecto. Se identifican restricciones en el proceso y el producto, y se traza un plan de gestión detallado. Se identifican los riesgos del proyecto. Pueden planearse estrategias alternativas, según sean los riesgos.
2. *Valoración y reducción del riesgo* En cada uno de los riesgos identificados del proyecto, se realiza un análisis minucioso. Se dan acciones para reducir el riesgo. Por ejemplo, si existe un riesgo de que los requerimientos sean inadecuados, puede desarrollarse un sistema prototipo.
3. *Desarrollo y validación* Después de una evaluación del riesgo, se elige un modelo de desarrollo para el sistema. Por ejemplo, la creación de prototipos desechables sería el mejor enfoque de desarrollo, si predominan los riesgos en la interfaz del usuario. Si la principal consideración son los riesgos de seguridad, el desarrollo con base en transformaciones formales sería el proceso más adecuado, entre otros. Si el principal riesgo identificado es la integración de subsistemas, el modelo en cascada sería el mejor modelo de desarrollo a utilizar.
4. *Planeación* El proyecto se revisa y se toma una decisión sobre si hay que continuar con otro ciclo de la espiral. Si se opta por continuar, se trazan los planes para la siguiente fase del proyecto.

La diferencia principal entre el modelo en espiral con otros modelos de proceso de software es su reconocimiento explícito del riesgo. Un ciclo de la espiral comienza por elaborar objetivos como rendimiento y funcionalidad. Luego, se numeran formas alternativas de alcanzar dichos objetivos y de lidiar con las restricciones en cada uno de ellos. Cada alternativa se valora contra cada objetivo y se identifican las fuentes de riesgo del proyecto. El siguiente paso es resolver dichos riesgos, mediante actividades de recopilación de información, como análisis más detallado, creación de prototipos y simulación.

Una vez valorados los riesgos se realiza cierto desarrollo, seguido por una actividad de planeación para la siguiente fase del proceso. De manera informal, el riesgo significa simplemente algo que podría salir mal. Por ejemplo, si la intención es usar un nuevo lenguaje de programación, un riesgo sería que los compiladores disponibles no sean confiables o no produzcan un código-objeto suficientemente eficaz. Los riesgos conducen a propuestas de cambios de software y a problemas de proyecto como exceso en las fechas y el costo, de manera que la minimización del riesgo es una actividad muy importante de administración del proyecto. En el capítulo 22 se tratará la gestión del riesgo, una parte esencial de la administración del proyecto.



2.4 El Proceso Unificado Racional

El Proceso Unificado Racional (RUP, por las siglas de *Rational Unified Process*) (Krutchen, 2003) es un ejemplo de un modelo de proceso moderno que se derivó del trabajo sobre el UML y el proceso asociado de desarrollo de software unificado (Rumbaugh *et al.*, 1999; Arlow y Neustadt, 2005). Aquí se incluye una descripción, pues es un buen ejemplo de un modelo de proceso híbrido. Conjunta elementos de todos los modelos de proceso genéricos (sección 2.1), ilustra la buena práctica en especificación y diseño (sección 2.2), y apoya la creación de prototipos y entrega incremental (sección 2.3).

El RUP reconoce que los modelos de proceso convencionales presentan una sola visión del proceso. En contraste, el RUP por lo general se describe desde tres perspectivas:

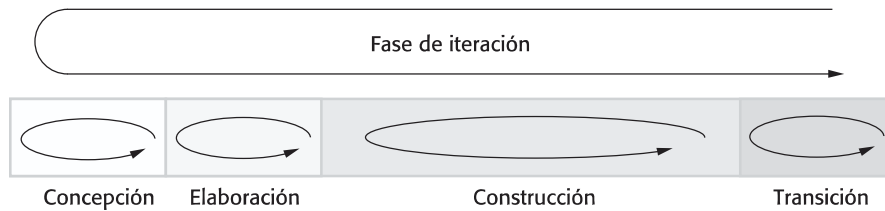
1. Una perspectiva dinámica que muestra las fases del modelo a través del tiempo.
2. Una perspectiva estática que presenta las actividades del proceso que se establecen.
3. Una perspectiva práctica que sugiere buenas prácticas a usar durante el proceso.

La mayoría de las descripciones del RUP buscan combinar las perspectivas estática y dinámica en un solo diagrama (Krutchen, 2003). Esto hace que el proceso resulte más difícil de entender, por lo que en este texto se usan descripciones separadas de cada una de estas perspectivas.

El RUP es un modelo en fases que identifica cuatro fases discretas en el proceso de software. Sin embargo, a diferencia del modelo en cascada, donde las fases se igualan con actividades del proceso, las fases en el RUP están más estrechamente vinculadas con la empresa que con las preocupaciones técnicas. La figura 2.11 muestra las fases del RUP. Éstas son:

1. *Concepción* La meta de la fase de concepción es establecer un caso empresarial para el sistema. Deben identificarse todas las entidades externas (personas y sistemas)

Figura 2.12 Fases en el Proceso Unificado Racional



que interactuarán con el sistema y definirán dichas interacciones. Luego se usa esta información para valorar la aportación del sistema hacia la empresa. Si esta aportación es menor, entonces el proyecto puede cancelarse después de esta fase.

2. *Elaboración* Las metas de la fase de elaboración consisten en desarrollar la comprensión del problema de dominio, establecer un marco conceptual arquitectónico para el sistema, diseñar el plan del proyecto e identificar los riesgos clave del proyecto. Al completar esta fase, debe tenerse un modelo de requerimientos para el sistema, que podría ser una serie de casos de uso del UML, una descripción arquitectónica y un plan de desarrollo para el software.
3. *Construcción* La fase de construcción incluye diseño, programación y pruebas del sistema. Partes del sistema se desarrollan en paralelo y se integran durante esta fase. Al completar ésta, debe tenerse un sistema de software funcionando y la documentación relacionada y lista para entregarse al usuario.
4. *Transición* La fase final del RUP se interesa por el cambio del sistema desde la comunidad de desarrollo hacia la comunidad de usuarios, y por ponerlo a funcionar en un ambiente real. Esto es algo ignorado en la mayoría de los modelos de proceso de software aunque, en efecto, es una actividad costosa y en ocasiones problemática. En el complemento de esta fase se debe tener un sistema de software documentado que funcione correctamente en su entorno operacional.

La iteración con el RUP se apoya en dos formas. Cada fase puede presentarse en una forma iterativa, con los resultados desarrollados incrementalmente. Además, todo el conjunto de fases puede expresarse de manera incremental, como se muestra en la flecha en curva desde *transición* hasta *concepción* en la figura 2.12.

La visión estática del RUP se enfoca en las actividades que tienen lugar durante el proceso de desarrollo. Se les llama flujos de trabajo en la descripción RUP. En el proceso se identifican seis flujos de trabajo de proceso centrales y tres flujos de trabajo de apoyo centrales. El RUP se diseñó en conjunto con el UML, de manera que la descripción del flujo de trabajo se orienta sobre modelos UML asociados, como modelos de secuencia, modelos de objeto, etcétera. En la figura 2.13 se describen la ingeniería central y los flujos de trabajo de apoyo.

La ventaja en la presentación de las visiones dinámica y estática radica en que las fases del proceso de desarrollo no están asociadas con flujos de trabajo específicos. En principio, al menos, todos los flujos de trabajo RUP pueden estar activos en la totalidad de las etapas del proceso. En las fases iniciales del proceso, es probable que se use mayor esfuerzo en los flujos de trabajo como modelado del negocio y requerimientos y, en fases posteriores, en las pruebas y el despliegue.

Flujo de trabajo	Descripción
Modelado del negocio	Se modelan los procesos de negocios utilizando casos de uso de la empresa.
Requerimientos	Se identifican los actores que interactúan con el sistema y se desarrollan casos de uso para modelar los requerimientos del sistema.
Análisis y diseño	Se crea y documenta un modelo de diseño utilizando modelos arquitectónicos, de componentes, de objetos y de secuencias.
Implementación	Se implementan y estructuran los componentes del sistema en subsistemas de implementación. La generación automática de código a partir de modelos de diseño ayuda a acelerar este proceso.
Pruebas	Las pruebas son un proceso iterativo que se realiza en conjunto con la implementación. Las pruebas del sistema siguen al completar la implementación.
Despliegue	Se crea la liberación de un producto, se distribuye a los usuarios y se instala en su lugar de trabajo.
Administración de la configuración y del cambio	Este flujo de trabajo de apoyo gestiona los cambios al sistema (véase el capítulo 25).
Administración del proyecto	Este flujo de trabajo de apoyo gestiona el desarrollo del sistema (véase los capítulos 22 y 23).
Entorno	Este flujo de trabajo pone a disposición del equipo de desarrollo de software, las herramientas adecuadas de software.

Figura 2.13 Flujos de trabajo estáticos en el Proceso Unificado Racional

El enfoque práctico del RUP describe las buenas prácticas de ingeniería de software que se recomiendan para su uso en el desarrollo de sistemas. Las seis mejores prácticas fundamentales que se recomiendan son:

1. *Desarrollo de software de manera iterativa* Incrementar el plan del sistema con base en las prioridades del cliente, y desarrollar oportunamente las características del sistema de mayor prioridad en el proceso de desarrollo.
2. *Gestión de requerimientos* Documentar de manera explícita los requerimientos del cliente y seguir la huella de los cambios a dichos requerimientos. Analizar el efecto de los cambios sobre el sistema antes de aceptarlos.
3. *Usar arquitecturas basadas en componentes* Estructurar la arquitectura del sistema en componentes, como se estudió anteriormente en este capítulo.
4. *Software modelado visualmente* Usar modelos UML gráficos para elaborar representaciones de software estáticas y dinámicas.
5. *Verificar la calidad del software* Garantizar que el software cumpla con los estándares de calidad de la organización.

6. *Controlar los cambios al software* Gestionar los cambios al software con un sistema de administración del cambio, así como con procedimientos y herramientas de administración de la configuración.

El RUP no es un proceso adecuado para todos los tipos de desarrollo, por ejemplo, para desarrollo de software embebido. Sin embargo, sí representa un enfoque que potencialmente combina los tres modelos de proceso genéricos que se estudiaron en la sección 2.1. Las innovaciones más importantes en el RUP son la separación de fases y flujos de trabajo, y el reconocimiento de que el despliegue del software en un entorno del usuario forma parte del proceso. Las fases son dinámicas y tienen metas. Los flujos de trabajo son estáticos y son actividades técnicas que no se asocian con una sola fase, sino que pueden usarse a lo largo del desarrollo para lograr las metas de cada fase.

PUNTOS CLAVE

- Los procesos de software son actividades implicadas en la producción de un sistema de software. Los modelos de proceso de software consisten en representaciones abstractas de dichos procesos.
- Los modelos de proceso general describen la organización de los procesos de software. Los ejemplos de estos modelos generales incluyen el modelo en cascada, el desarrollo incremental y el desarrollo orientado a la reutilización.
- La ingeniería de requerimientos es el proceso de desarrollo de una especificación de software. Las especificaciones tienen la intención de comunicar las necesidades de sistema del cliente a los desarrolladores del sistema.
- Los procesos de diseño e implementación tratan de transformar una especificación de requerimientos en un sistema de software ejecutable. Pueden usarse métodos de diseño sistemáticos como parte de esta transformación.
- La validación del software es el proceso de comprobar que el sistema se conforma a su especificación y que satisface las necesidades reales de los usuarios del sistema.
- La evolución del software tiene lugar cuando cambian los sistemas de software existentes para satisfacer nuevos requerimientos. Los cambios son continuos y el software debe evolucionar para seguir siendo útil.
- Los procesos deben incluir actividades para lidiar con el cambio. Esto puede implicar una fase de creación de prototipos que ayude a evitar malas decisiones sobre los requerimientos y el diseño. Los procesos pueden estructurarse para desarrollo y entrega iterativos, de forma que los cambios se realicen sin perturbar al sistema como un todo.
- El Proceso Unificado Racional es un modelo de proceso genérico moderno que está organizado en fases (concepción, elaboración, construcción y transición), pero separa las actividades (requerimientos, análisis y diseño, etcétera) de dichas fases.

LECTURAS SUGERIDAS

Managing Software Quality and Business Risk. Aun cuando éste es principalmente un libro sobre administración de software, incluye un excelente capítulo (capítulo 4) de modelos de proceso. (M. Ould, John Wiley and Sons Ltd, 1999.)

Process Models in Software Engineering. Ofrece una excelente visión de un amplio rango de modelos de proceso de ingeniería de software que se han propuesto. (W. Scacchi, *Encyclopaedia of Software Engineering*, ed. J.J. Marciniak, John Wiley and Sons, 2001.)
<http://www.ics.uci.edu/~wscacchi/Papers/SE-Encyc/Process-Models-SE-Encyc.pdf>.

The Rational Unified Process—An Introduction (3rd edition). Éste es el libro más legible que hay disponible sobre RUP hasta ahora. Krutchen describe bien el proceso, pero sería más deseable ver las dificultades prácticas de usar el proceso. (P. Krutchen, Addison-Wesley, 2003.)

EJERCICIOS

- 2.1. Explicando las razones para su respuesta, y con base en el tipo de sistema a desarrollar, sugiera el modelo de proceso de software genérico más adecuado que se use como fundamento para administrar el desarrollo de los siguientes sistemas:
 - Un sistema para controlar el antibloqueo de frenos en un automóvil
 - Un sistema de realidad virtual para apoyar el mantenimiento de software
 - Un sistema de contabilidad universitario que sustituya a uno existente
 - Un sistema interactivo de programación de viajes que ayude a los usuarios a planear viajes con el menor impacto ambiental
- 2.2. Explique por qué el desarrollo incremental es el enfoque más efectivo para diseñar sistemas de software empresariales. ¿Por qué este modelo es menos adecuado para ingeniería de sistemas de tiempo real?
- 2.3. Considere el modelo de proceso basado en reutilización que se muestra en la figura 2.3. Explique por qué durante el proceso es esencial tener dos actividades separadas de ingeniería de requerimientos.
- 2.4. Sugiera por qué, en el proceso de ingeniería de requerimientos, es importante hacer una distinción entre desarrollar los requerimientos del usuario y desarrollar los requerimientos del sistema.
- 2.5. Describa las principales actividades en el proceso de diseño de software y las salidas de dichas actividades. Con un diagrama, muestre las posibles relaciones entre las salidas de dichas actividades.
- 2.6. Explique por qué el cambio es inevitable en los sistemas complejos, y mencione ejemplos (además de la creación de prototipos y la entrega incremental) de las actividades de proceso de software que ayudan a predecir los cambios y a lograr que el software por desarrollar sea más resistente al cambio.

- 2.7. Explique por qué los sistemas desarrollados como prototipos por lo general no deben usarse como sistemas de producción.
- 2.8. Exponga por qué el modelo en espiral de Boehm es un modelo adaptable que puede apoyar las actividades tanto de evitar el cambio como de tolerar el cambio. En la práctica, este modelo no se ha usado ampliamente. Sugiera por qué éste podría ser el caso.
- 2.9. ¿Cuáles son las ventajas de proporcionar visiones estática y dinámica del proceso de software como en el Proceso Unificado Racional?
- 2.10. Históricamente, la introducción de la tecnología ha causado profundos cambios en el mercado laboral y, al menos temporalmente, ha reemplazado a personas en los puestos de trabajo. Explique si es probable que la introducción de extensos procesos de automatización tenga las mismas consecuencias para los ingenieros de software. Si no cree que haya consecuencias, explique por qué. Si cree que reducirá las oportunidades laborales, ¿es ético que los ingenieros afectados resistan pasiva o activamente la introducción de esta tecnología?

REFERENCIAS

- Arlow, J. y Neustadt, I. (2005). *UML 2 and the Unified Process: Practical Object-Oriented Analysis and Design (2nd Edition)*. Boston: Addison-Wesley.
- Boehm, B. y Turner, R. (2003). *Balancing Agility and Discipline: A Guide for the Perplexed*. Boston: Addison-Wesley.
- Boehm, B. W. (1988). "A Spiral Model of Software Development and Enhancement". *IEEE Computer*, **21** (5), 61–72.
- Budgen, D. (2003). *Software Design (2nd Edition)*. Harlow, UK.: Addison-Wesley.
- Krutchén, P. (2003). *The Rational Unified Process—An Introduction (3rd Edition)*. Reading, MA: Addison-Wesley.
- Massol, V. y Husted, T. (2003). *JUnit in Action*. Greenwich, Conn.: Manning Publications Co.
- Rettig, M. (1994). "Practical Programmer: Prototyping for Tiny Fingers". *Comm. ACM*, **37** (4), 21–7.
- Royce, W. W. (1970). "Managing the Development of Large Software Systems: Concepts and Techniques". IEEE WESTCON, Los Angeles CA: 1–9.
- Rumbaugh, J., Jacobson, I. y Booch, G. (1999). *The Unified Software Development Process*. Reading, Mass.: Addison-Wesley.
- Schmidt, D. C. (2006). "Model-Driven Engineering". *IEEE Computer*, **39** (2), 25–31.
- Schneider, S. (2001). *The B Method*. Houndmills, UK: Palgrave Macmillan.
- Wordsworth, J. (1996). *Software Engineering with B*. Wokingham: Addison-Wesley.